ITE 18 – Application Development and Emerging Technologies

Web/Mobile Application - Final Project Submission

Student Name(s): Daniel D. Olofernes

Project Title: TechStore E-Commerce

Date of Submission:12/26/2025

Course/Section: ITE-18

---

1. Project Overview

1.1 Project Description

Provide a brief overview of your application. What problem does it solve? What is its purpose?

TechStore is a comprehensive full-stack e-commerce application designed to provide businesses with a complete online retail solution. The application handles the entire customer lifecycle from product browsing and purchasing to order management, returns processing, and payment handling.

Problem Solved: Traditional e-commerce platforms often lack integrated solutions for order management, return processing, and promotional campaigns. TechStore combines these critical features into a single, unified platform that serves both customers and administrators seamlessly.

Purpose: To create a modern, scalable e-commerce platform that enables businesses to:

- Manage product catalogs and inventory
- Process customer orders securely
- Handle returns and refunds efficiently
- Run promotional campaigns with vouchers
- Provide customers with order tracking and management tools
- Enable administrators with comprehensive dashboards and controls

1.2 Target Users

Who are the intended users of your application? Describe their characteristics and needs.

Primary Users:

1.Customer/Shoppers Characteristics: Online shoppers of technology products Needs browse products, place orders, track purchases, manage returns, apply discount codes.

2. Administrators/Store Managers Characteristics: Business owner managing store operations Needs to Manage inventory, process orders, approve returns, distribute vouchers.

1.3 Key Features

List the main features and functionalities of your application.

- Feature 1: **Product Management** - Browse, search, and filter technology products with detailed information
- Feature 2: **Shopping Cart & Checkout** - Add items to cart with secure checkout process
- Feature 3: **Order Management** - Users can view all their orders with detailed status tracking and item breakdowns
- Feature 4: **Payment Processing** - Multiple payment method support with secure transaction handling
- Feature 5: **Return Management System** - Customers can request returns with automatic refund calculation and admin approval workflow
- Feature 6: **Voucher Distribution** - Admins can create and distribute discount vouchers with flexible conditions (percentage or fixed amount)
- Feature 7: **Admin Dashboard** - Comprehensive administrative interface for managing: Customer orders and order items Return requests with approval/rejection workflows Voucher creation and distribution Order completion tracking
- Feature 8: **User Order Dashboard** - Customers view their complete order history with statistics (total orders, total spent, pending orders)
- Feature 9: **Order Details** - Detailed views of individual orders with items, pricing.

1.4 Technology Stack

List all technologies, frameworks, libraries, and tools used in your project.

Frontend: Vue.js/Nuxt.js, vite, Tailwind CSS 4.0, Axios
Backend: Laravel, Laravel sanctum
Database: MySQL
Other Tools: Figma, Postman

2. App Plan

2.1 Project Scope

Define what is included and what is not included in this project.

In Scope:

- Core E-Commerce Features:

Product catalog management with brands and categories. Customer registration and authentication (user & admin). Product browsing, searching, and filtering Shopping, cart functionality, order creation and processing, order item tracking and management Payment processing and transaction records, User order history and dashboard.

- Return Management System: Customer-initiated return requests with itemized selection Return request creation form with reason selection Return request dashboard with filtering and statistics Admin review and approval/rejection workflow Automatic refund calculation of the Return request item
- Voucher Distribution System: Admin voucher creation with flexible discount types (percentage or fixed amount) Automatic voucher distribution to all users Batch voucher redistribution to new users Voucher usage tracking and statistics Voucher expiration date management Admin dashboard for voucher management
- Customer Features: User dashboard displaying all orders Order details view with item breakdown Order statistics (total orders, total spent, pending orders) Return request submission and tracking Return request history and status monitoring Voucher application at checkout Account management
- API Endpoints: RESTful API for customers, products, orders, order items, shipping, vouchers, and cart Authentication endpoints (login, register) Public and authenticated endpoint access control JSON response formatting.
- Frontend User Interface: Responsive web design Blade template rendering for server-side views Nuxt.js/Vue.js components for frontend interactions Tailwind CSS styling Form validation and error handling Real-time statistics and dashboards.

Out of Scope:

- Customer review and rating system
- Product variant management (size, color, etc.)

2.2 Objectives & Goals

State the specific, measurable objectives for this project.

1. Deliver an E-Commerce Platform Successfully implement a fully functional online retail system capable of handling product sales Measurable: All core features (products, orders, payments) operational and tested
2. Implement Comprehensive Order Management System Enable customers to view, track, and manage their orders seamlessly Enable admins to review and process orders efficiently Measurable: 100% order lifecycle features working (creation, viewing) Success Criteria: Users can view all orders.
3. Build Functional Return Request Management Create a complete return workflow from customer request to admin approval Implement automatic refund calculations and tracking Measurable: Return requests can be created, approved/rejected, and tracked Success Criteria: Full audit trail of return status changes.
4. Develop Admin Voucher Distribution System Enable admins to create and distribute promotional vouchers Implement automatic batch distribution to users Measurable: Admins can create, edit, delete, and redistribute vouchers with full tracking Success Criteria: Vouchers appear on all user accounts and can be applied at checkout.
5. Create User-Friendly Admin Dashboard Build comprehensive admin interfaces for managing orders, returns, and vouchers Provide real-time statistics and filtering capabilities Measurable: All admin features accessible from sidebar with clear navigation Success Criteria: Admin can manage 3+ major feature areas from dashboard.
6. Ensure Data Security and Authorization Implement role-based access control (customer vs admin) Protect all sensitive operations with proper authentication Measurable: No unauthorized access to other users' data Success Criteria: All endpoints secured with appropriate middleware

2.3 User Stories & Use Cases

Describe key user stories in the format: "As a [user type], I want to [action], so that [benefit]" **User Story 1:**

- As a customer, I want to browse the product catalog, view product details, add items to my cart, and complete a purchase, so that I can find and buy technology products conveniently.
- Acceptance Criteria:
    - Criterion 1: Customer can view all available products with descriptions and prices
    - Criterion 2: Customer can add products to their shopping cart
    - Criterion 3: Customer can proceed to checkout and complete a purchase

- Criterion 4: Customer can filter/search products by category or brand **User Story 2:**

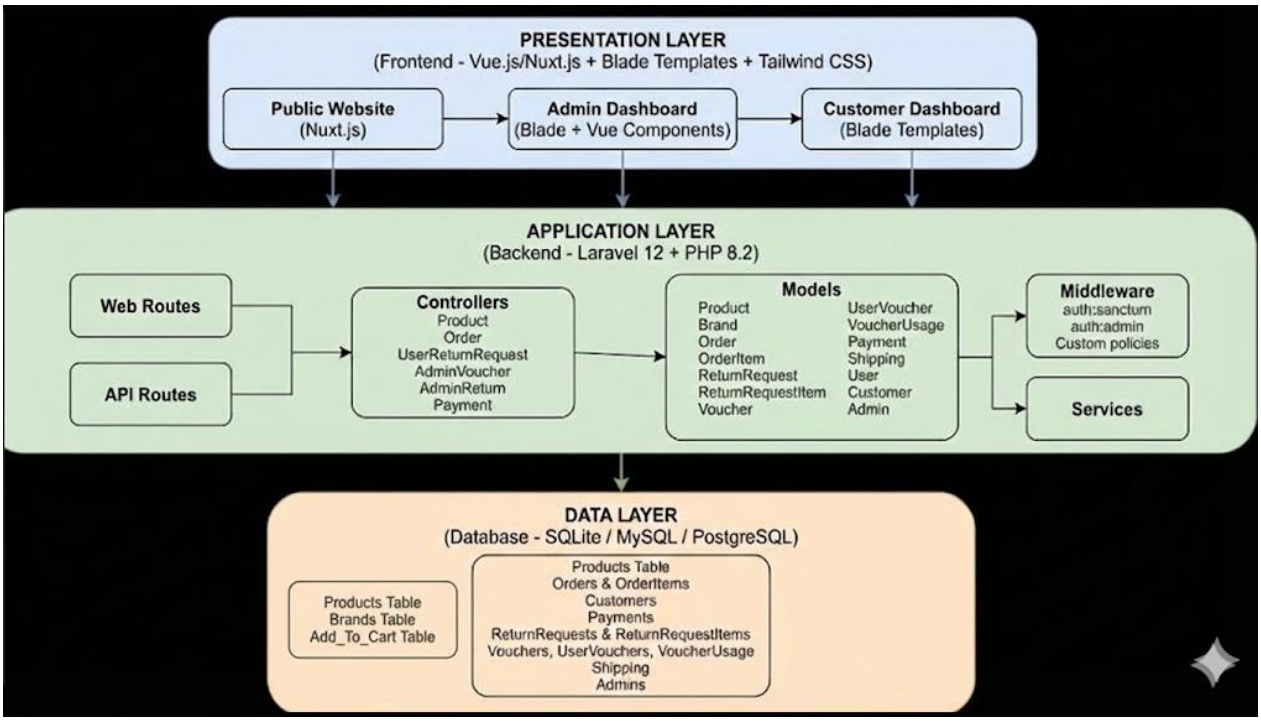
- As a customer, I want to view all my past orders, see detailed information about each order, and track their status, so that I can monitor my purchases and know when they will arrive.
- Acceptance Criteria:
  - Criterion 1: Customer can access their order history dashboard from navigation menu
  - Criterion 2: Customer can see a list of all their orders with key information (date, total, status)
  - Criterion 3: Customer can click on an order to view detailed information including all items purchased
  - Criterion 4: Customer can see order statistics (total orders, total spent, pending orders)

User Story 3: Admin Reviews and Approves Return Request

- As an admin, I want to review all customer return requests, view item details and reasons, and approve or reject requests, So that I can manage returns efficiently and process refunds appropriately.
- Acceptance Criteria:
  - Admin can access the returns dashboard from the admin sidebar
  - Admin can see a list of all return requests with customer and status information
  - Admin can filter return requests by status (pending, approved, rejected, refunded)

2.4 System Architecture

Describe the overall architecture of your application. You may include a diagram or flowchart.



Component interactions

1. **Frontend Initiation**

- User interacts with Nuxt.js frontend or accesses Blade templates

- Requests sent via HTTP/HTTPS to the backend

2. **Routing**

- Laravel router directs requests to appropriate controllers

- Two types of routes:

- Web routes: Return Blade-rendered HTML pages

- API routes: Return JSON responses for Nuxt.js

3. **Authentication & Authorization**

- Sanctum validates API tokens for authenticated requests

- Middleware checks user role

- Routes protected with auth guards

4. **Controller Logic**

- Controllers handle business logic and data processing

- Call Model methods for database operations

- Implement validation and error handling

5. **Database Operations**

- Eloquent ORM translates PHP to SQL queries

- Models establish relationships between entities

- Transactions ensure data integrity for complex operations

6. **Response Generation**

- Controllers return JSON responses for API

- Frontend renders or displays responses to user

2.5 Development Timeline

Outline your development phases and estimated timelines.

Phase	Description	Timeline
Phase 1	Design & Planning	Week 1-2
Phase 2	Database Setup	Week 2-3
Phase 3	Frontend Development	Week 3-7
Phase 4	Backend Development	Week 3-7
Phase 5	Integration & Testing	Week 5-7

---

3. UI/UX Design

3.1 Design Philosophy

Explain your design approach and the UI/UX principles you applied.

TechStore E-Commerce follows a **modern, user-centric design approach** focused on:

- Simplicity: Minimize cognitive load with intuitive navigation and clear information hierarchy

- Consistency: Unified visual language across all pages and components

- Performance: Fast loading times and smooth interactions

Design Principles Applied:

1. User-Centered Design - Every feature designed with user needs and workflows

2. Visual Hierarch - Important information emphasized through size, color, and position

3. Feedback & Confirmation - Clear messages for user actions and system responses

5. Consistency - Familiar patterns reduce learning curve for users

6. Error Prevention - Validation and confirmations prevent mistakes

7. Aesthetic Integrity - Design supports and enhances usability

3.2 Color Scheme

Describe the colors used in your application and their purpose.

- **Primary Color: Blue (#007BFF)** - Used for Primary buttons, links, active states, and interactive elements
- **Secondary Color: Gray (#6C757D)** - Used for [purpose]
- **Accent Color: Green (#28A745)** - Used for Secondary buttons, inactive states, disabled content
- **Background Color: White (#FFFFFF)**
- **Text Color: Dark Gray (#212529)**

3.3 Typography

Describe the fonts and text styles used.

- **Font Family:** Inter, -apple-system, BlinkMacSystemFont, Segoe UI, sans-serif
- **Heading Font Size:** 16px, 20px, 24px, 32px
- **Body Font Size:** 14px, 16px
- **Line Height:** 1.2, 1.3, 1.4, 1.5, 1.6

3.4 UI Components

List and describe the key UI components used in your application.

- **Button:**
    - Primary Button (Blue) - Usage: Main call-to-action buttons - Padding: 12px 24px (44px minimum height for touch) - Border Radius: 6px ○ Secondary Button (Gray) - Usage: Alternative actions, less prominent - Padding: 10px 20px - Border Radius: 6px ○

Danger Button (Red) - Usage: Destructive actions requiring confirmation - Padding: 12px 24px - Border Radius: 6px

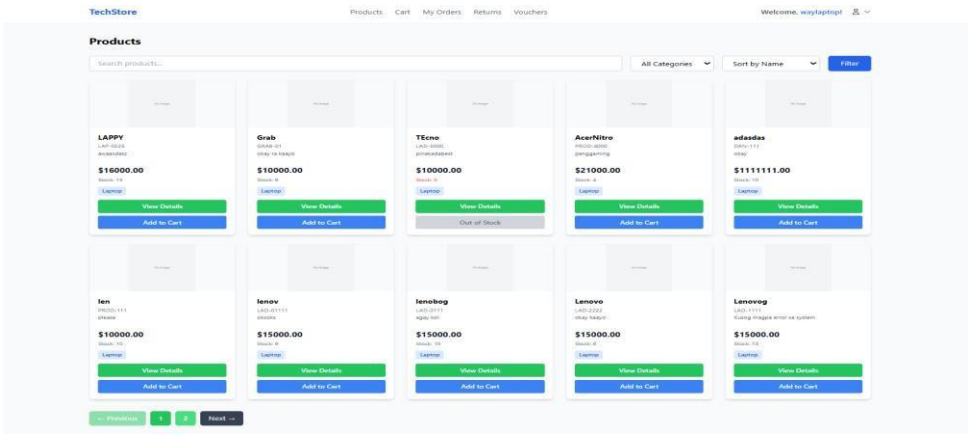
- **Input Fields:**
  - Text Input - Border: 1px solid #DEE2E6 - Padding: 10px 12px - Border Radius: 6px - Focus State: Blue border (#007BFF), subtle box-shadow - Placeholder: Light gray (#6C757D) ○ Form Validation: - Valid: Green left border with checkmark icon - Invalid: Red left border with error icon - Error Message: Red text below field (12px, 0.75rem)
  - Input Types Supported: - Text, Email, Password, Number, Date, Textarea - Autocomplete hints for enhanced UX - Clear button (X) for text fields
- **Navigation Bar:**
  - Structure: - Height: 60px (responsive, collapses to hamburger on mobile) - Background: White with subtle shadow - Logo/Brand: Left side, 24px x 24px - Menu Items: Center alignment, hover underline - User Menu: Right side, dropdown on click
  - Menu Items:** - Home, Products, About, Contact - Authenticated users: My Orders, Returns, Account - Admin users: Admin Dashboard, Reports - Login/Logout buttons
- **Cards:**
  - Product Card** - Background: White - Border: 1px solid #DEE2E6 - Border Radius: 8px - Padding: 16px - Hover Effect: Subtle shadow elevation, slight scale - Content: Image, name, price, rating, "Add to Cart" button ○ Order Card** - Background: White - Border-left: 4px solid (color by status) - Padding: 16px - Content: Order ID, date, total, status badge, "View Details" link - Hover Effect: Background color change, cursor pointer ○ Stat Card** (Dashboard) - Background: Gradient (light to white) - Border Radius: 8px - Icon: Large, left-aligned - Title: Small label - Value: Large bold number - Comparison: Previous period comparison percentage ○ Status Cards** (Return/Voucher Details) - Colored header bar matching status color - Icon in header - Title and description - Related actions buttons
- **Modal/Dialog:**
  - Confirmation Dialog** - Overlay: Semi-transparent dark background (rgba(0, 0, 0, 0.5)) - Modal: White background, centered, max-width 500px - Header: Title, close button - Body: Message and context - Footer: Cancel button (left), Confirm button (right) - Animation: Fade in/slide down on open
- **Alert Messages:**

- Success Alert** - Background: Light green (#D4EDDA) - Border-left: 4px solid green (#28A745) - Icon: Checkmark - Text: Dark green color - Dismissible: X button ○ Error Alert** - Background: Light red (#F8D7DA) - Border-left: 4px solid red (#DC3545) - Icon: X or exclamation - Text: Dark red color - Auto-dismiss: 5 seconds (optional)

- **Pagination:**
  - Component Structure:** - Previous/Next buttons - Numbered page links (showing 5 pages max) - Current page: Highlighted in blue - Ellipsis: For skipped pages - Disabled state: Previous on page 1, Next on last page

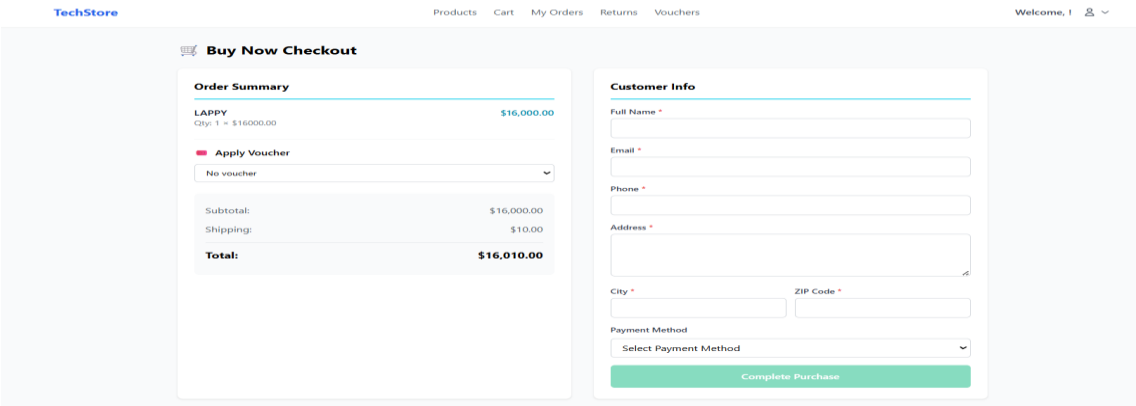
3.5 Wireframes & Mockups

Include screenshots or links to your wireframes and high-fidelity mockups for each main screen. **Screen 1: User Product Dashboard**



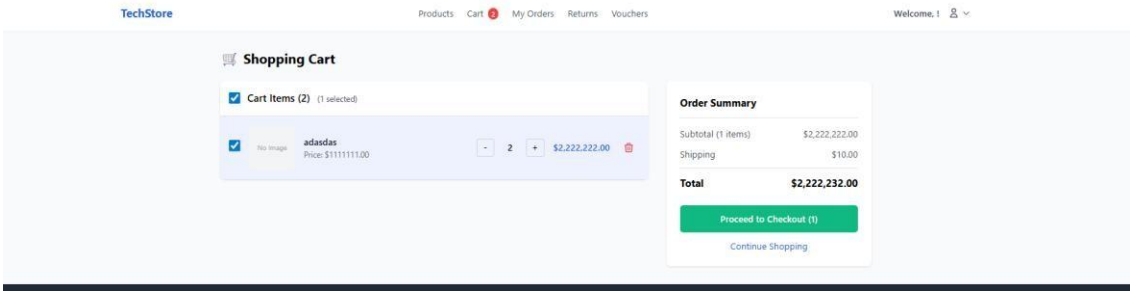
-In this page the user can see the products available and they can view the details and add it to their cart and also checkout the items

Screen 2: User Checkout page



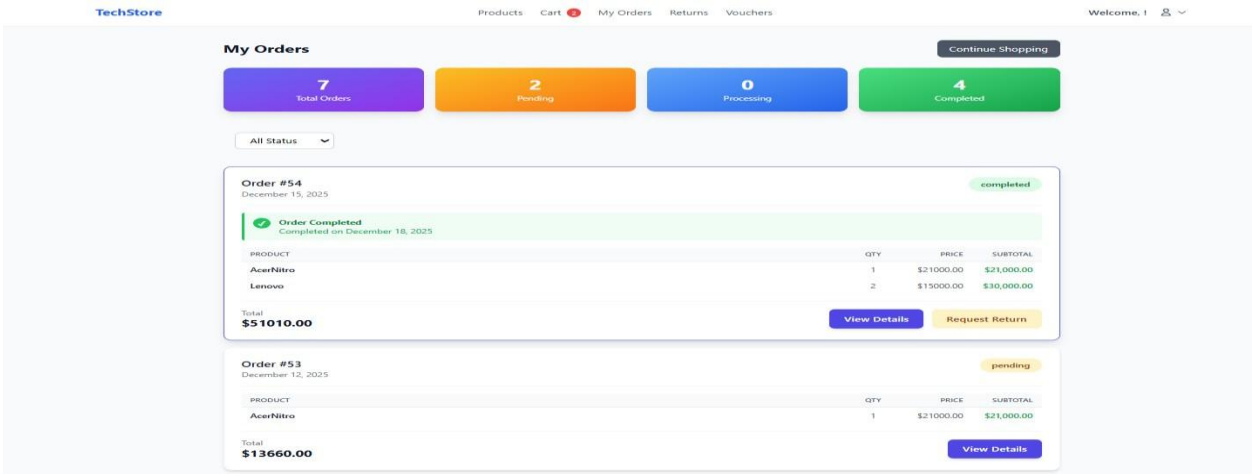
-This page will show the checkout process where the user input the information and select a payment method

Screen 3: User Cart



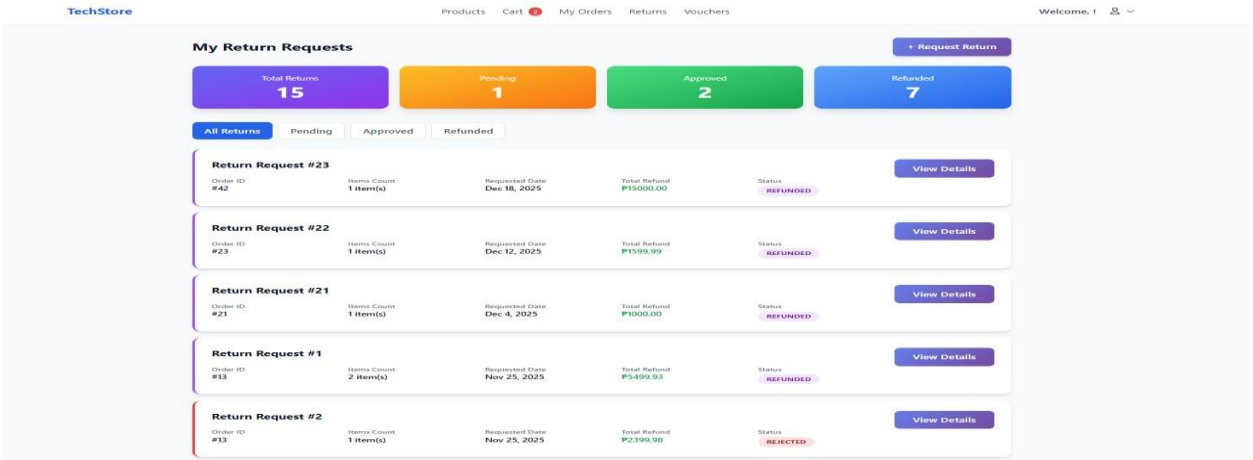
-In this page the user can see the products the added to cart and also they can select what product they wanted to continue/proceed to checkout

Screen 4: My Orders



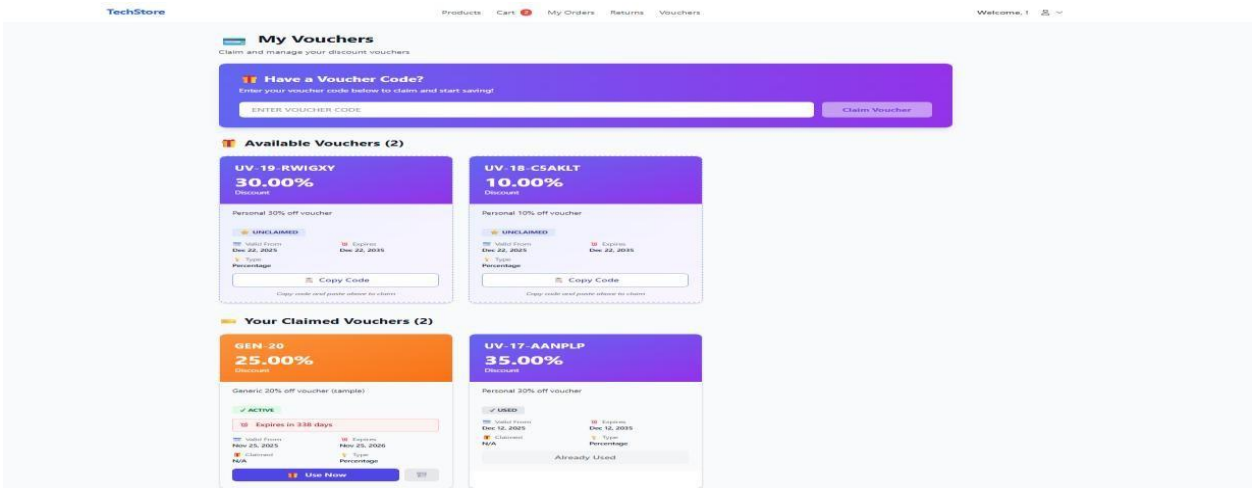
-This Page the user can see what product did he placed order and also the status of the order

Screen 5: Return Request



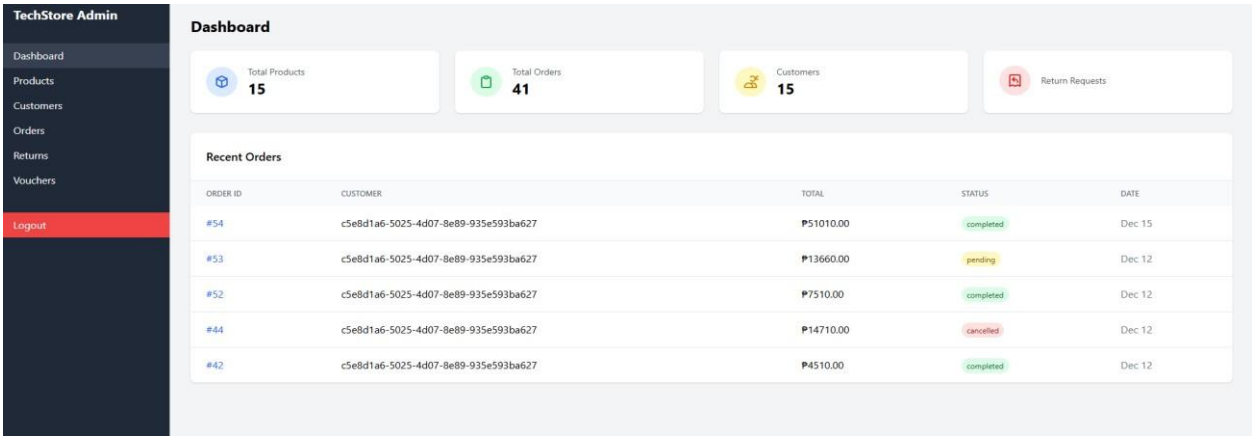
-This page the user can return the product if it's damage during the shipment or wrong item received

Screen 6: My Voucher



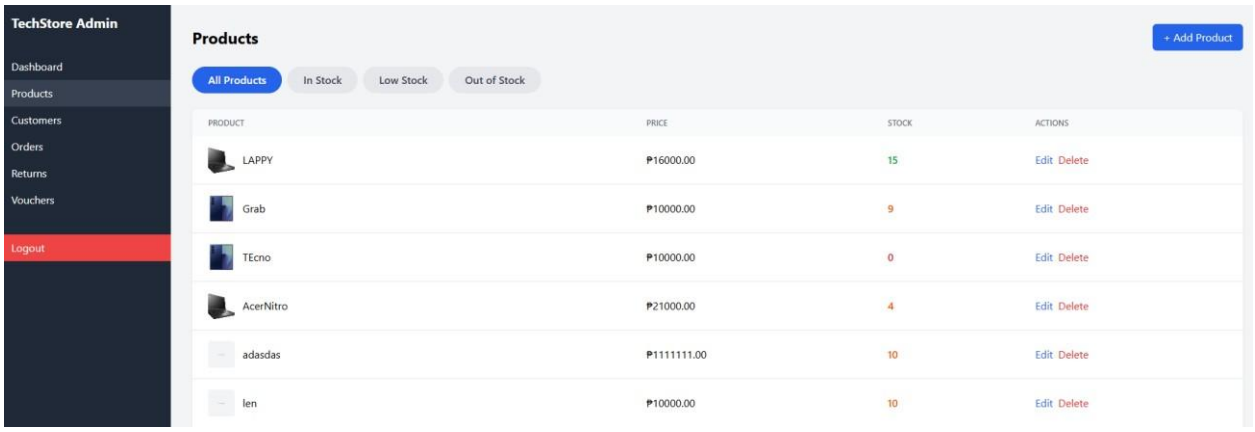
-This page the user can see the available voucher and the voucher that the user can be used

Screen 7: Admin Dashboard



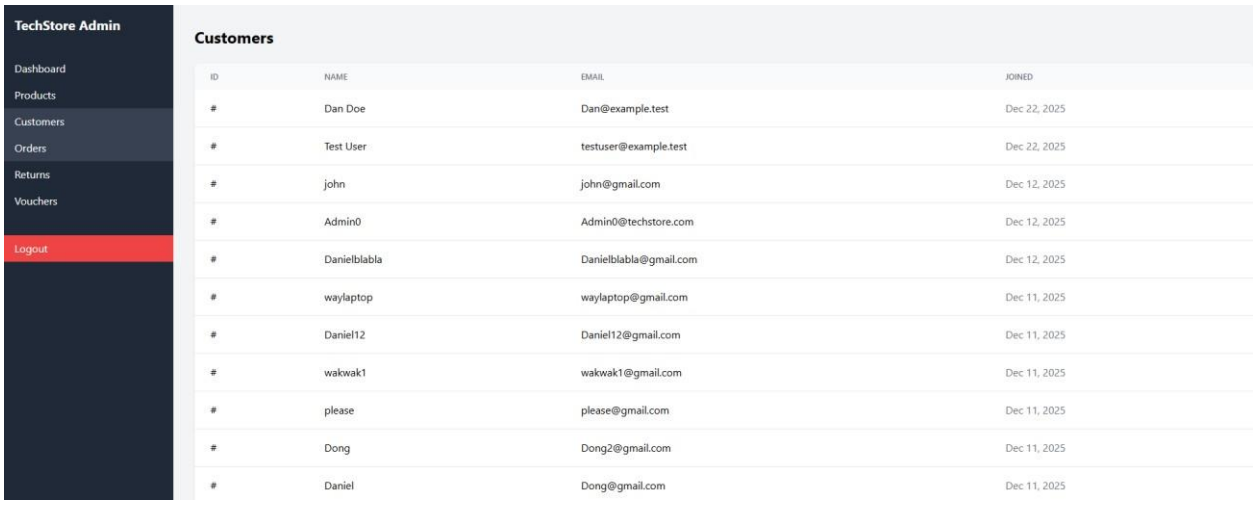
-In this page the admin dashboard will display the recent orders

Screen 8: Admin Products Management



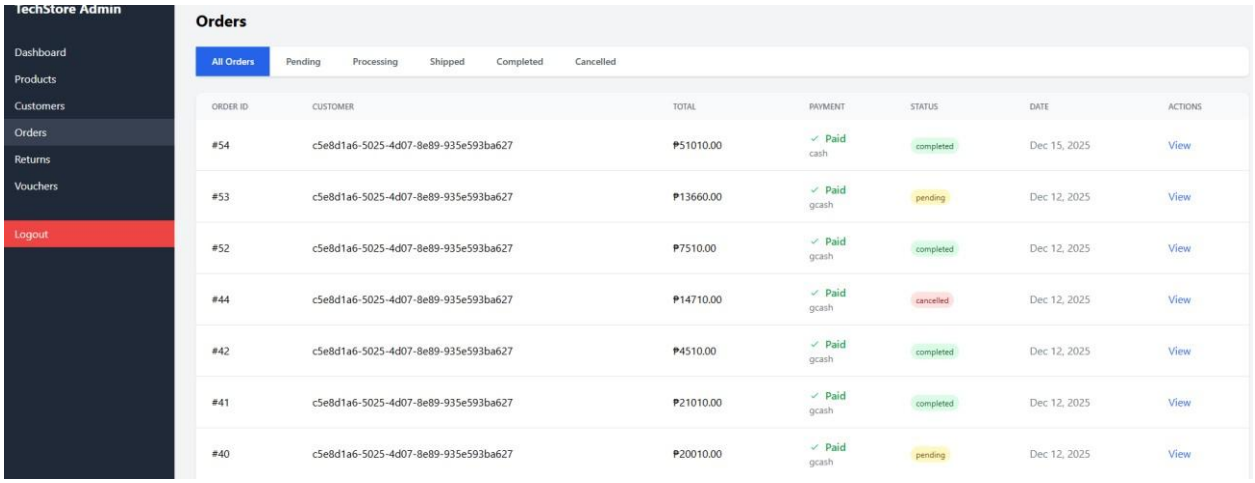
-In this page the admin can see the products list the price and there's a button for to determine the stocks of products also the admin can add a new product and edit the products that has been already added

Screen 9: Customer Dasboard



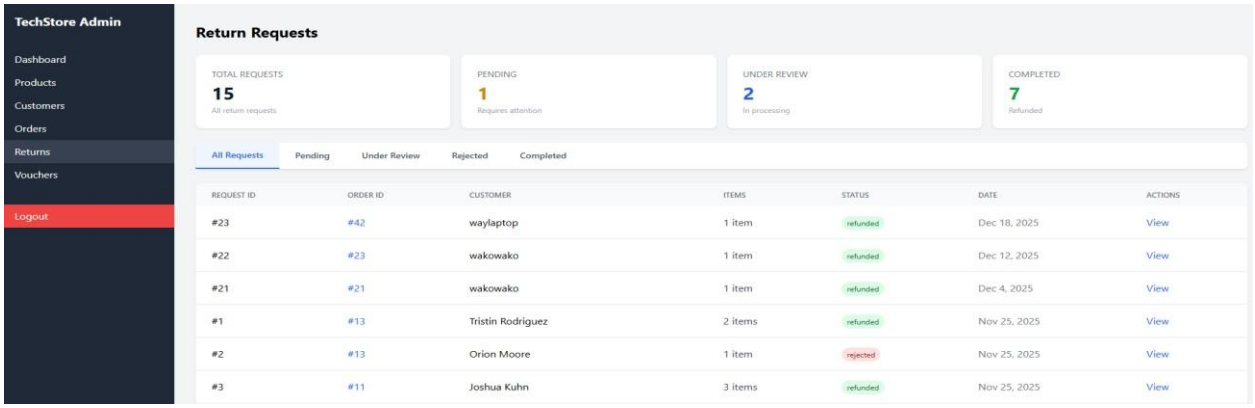
In this page the it will display the list of the customers details that registered into the web Screen 8:

Scene 10: Orders Management



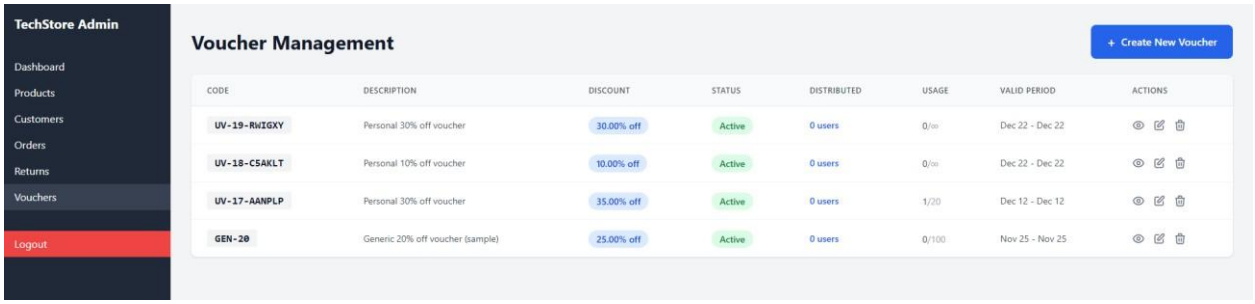
-In this page the admin can see the order details of the products of the customers purchased and also the admin can will manage the order if the order is paid through digital payment the admin can mark the order as completed

Screen 11: Return request management



-In this page the admin can review the return request items that the customer created and the admin will approved/reject the request depends on the reason

Screen 12: Voucher



-in this page the admin will create a voucher and distribute it to the customer also the admin can set the voucher limit that only 20 customer can claim it. Also the admin can set the distribution to active/inactive.

3.6 User Flows

Describe the main user flows through your application. Include diagrams if possible.

Flow 1: User Registration

START

↓

User visits application → Not authenticated

↓

Click "Register" button

↓

Enter email, password, confirm password

↓

System validates input

├– Invalid? → Display error message → Back to form

└– Valid? → Continue

↓

System checks email uniqueness

├– Email exists? → Display error → Back to form

└– New email? → Continue

↓

Hash password securely

↓

Create User record in database

↓

Generate Sanctum API token

↓

Store token in session/localStorage

↓

Redirect to dashboard

↓

User authenticated ✓

↓

END

Flow 2: Making a Purchase

START

↓

User browses products

↓

↓

Click "Add to Cart" on product

Quantity validation

├– Invalid stock? → Display error

└ Valid? → Continue

↓

Add item to shopping cart

↓

Update cart count in navbar

↓

User continues shopping OR clicks "Proceed to Checkout"

↓

View cart page

├– Modify quantities

├– Remove items

└ Review total

↓

Click "Checkout"

↓

Verify user authentication

├– Not logged in? → Redirect to login

└ Logged in? → Continue

↓

Enter/confirm shipping address

↓

Select shipping method

↓

Apply voucher code (optional)

↓

├– Valid code? → Calculate discount

└– Invalid? → Display error

Review order summary

↓

Enter payment information

↓

Process payment

├– Payment failed? → Display error, retry

└– Payment successful? → Continue

↓

Create Order record with status "Paid"

↓

Create OrderItem records for each cart item

↓

Mark UserVoucher as used (if applicable)

↓

Clear shopping cart

↓

Send order confirmation email

↓

Redirect to order confirmation page

↓

Order created ✓

↓

END

FLOW 3: **Requesting A Return**

↓

START

↓

User authenticated

↓

Click "Returns" in navigation

↓

View return requests dashboard

├– Existing returns displayed

├– Statistics shown

└– Filter by status

↓

Click "Request Return" button

↓

Select order from dropdown

↓

System displays order items

↓

Select specific items to return

├– Check item checkboxes

└– Set quantity for each

↓

For each item: Select return reason

├– Options: Defective, Wrong Item, Not as Described, Changed Mind, Other

└– Add optional notes

↓

System calculates refund amount

├– Based on item prices

└– Display total refund

↓

Review return request

↓

Click "Submit Return Request"

↓

Validation

├─ All required fields? → Continue

└─ Missing fields? → Display error

↓

Create ReturnRequest record with status "Pending"

↓

Create ReturnRequestItem records for each item

↓

Send confirmation email to customer

↓

Redirect to return details page

↓

Return created ✓

↓

END

FLOW 4: **Admin Approving a Return Request**

START

↓

Admin logs in

↓

Click "Returns" in sidebar

↓

View admin returns dashboard

├─ All return requests listed

├─ Filter by status

└─ Statistics displayed

↓

Click on return request to view details

↓

Review return information

├─ Customer details

├─ Order information

├─ Items being returned

├─ Reasons provided

└─ Refund amount

↓

Admin decision

├─ Click "Approve"?

| ├─ Enter optional notes

| └─ Continue to approve

├─ Click "Reject"?

| ├─ Enter rejection reason (required)

| └─ Continue to reject

└─ Leave pending? → End flow

↓

Process decision

├─ Approve: Update status to "approved"

├─ Reject: Update status to "rejected"

└─ Record admin_id and timestamp

↓

Create status change log entry

↓

Send notification email to customer

├– Approved: Include return shipping label info

└– Rejected: Include reason

↓

Update dashboard statistics

↓

Return processed ✓

↓

END

FLOW 5: **Admin Creating and Distributing Vouchers**

START

↓

Admin logs in

↓

Click "Voucher Management" in sidebar

↓

View vouchers list

├– All vouchers displayed

├– Statistics shown

└– Filter options available

↓

Click "Create New Voucher"

↓

Enter voucher details

├– Code (e.g., SUMMER20)

├– Description

├– Discount type (percentage or fixed)

├– Discount value

├– Start date

├– End date

├– Usage limit

└– Status (active/inactive)

↓

System validates input

├– Valid? → Continue

└– Invalid? → Display error

↓

Check code uniqueness

├– Code exists? → Display error

└– Unique? → Continue

↓

Create Voucher record

↓

System auto-distributes to all users

├– Get all User records

├– Create UserVoucher for each user

└– Batch insert for efficiency

↓

Send email to all users with code

↓

Display success message

├– Quantity distributed shown

└– Confirmation details

↓

Redirect to voucher details page

↓

Voucher created & distributed ✓

↓

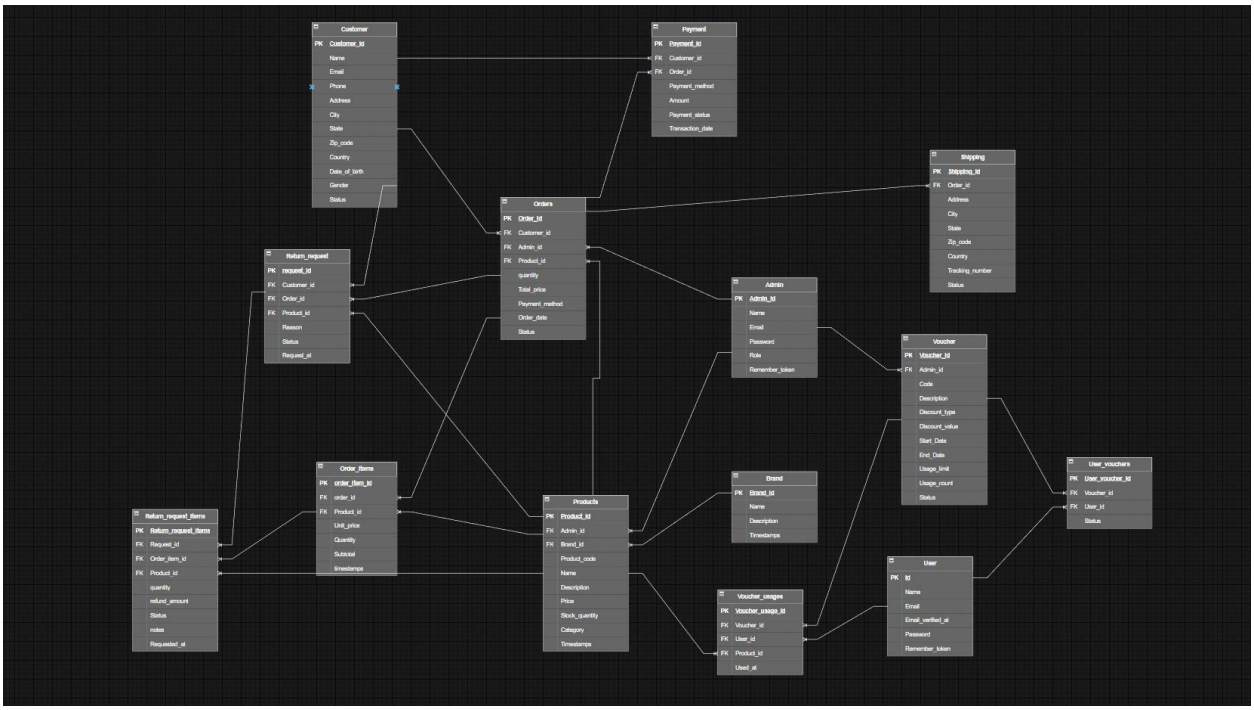
END

---

4. Database Architecture (ERD)

4.1 Entity Relationship Diagram

Include your ERD diagram here. Show all entities, attributes, and relationships.



4.2 Entity Descriptions

Provide detailed descriptions of each entity in your database.

Entity 1: USERS

| Field | Data Type | Constraints | Description ||-------|-----------|-------------|-------------|

| id | BIGINT | PRIMARY KEY, AUTO_INCREMENT | Unique user identifier |

| name | VARCHAR(255) | NOT NULL | User's full name |

| email | VARCHAR(255) | NOT NULL, UNIQUE | User's email address |

| email_verified_at | TIMESTAMP | NULLABLE | Email verification timestamp |

| password | VARCHAR(255) | NOT NULL | Hashed password |

| remember_token | VARCHAR(100) | NULLABLE | Remember me token |

| created_at | TIMESTAMP | NOT NULL | Account creation timestamp |

| updated_at | TIMESTAMP | NOT NULL | Last update timestamp |

Entity 2: ADMIN

| Field | Data Type | Constraints | Description |

|-------|-----------|-------------|-------------|

| admin_id | BIGINT | PRIMARY KEY, AUTO_INCREMENT | Unique admin identifier |

| name | VARCHAR(255) | NOT NULL | Admin's full name |

| email | VARCHAR(255) | NOT NULL, UNIQUE | Admin's email address |

| password | VARCHAR(255) | NOT NULL | Hashed password |

| role | ENUM('admin') | DEFAULT 'admin' | Admin role (extensible for future) |

| remember_token | VARCHAR(100) | NULLABLE | Remember me token |

| created_at | TIMESTAMP | NOT NULL | Account creation timestamp |

| updated_at | TIMESTAMP | NOT NULL | Last update timestamp |

Entity 3: PRODUCTS

|Field | Data Type | Constraints | Description |

|-------|-----------|-------------|-------------|

| id | BIGINT | PRIMARY KEY, AUTO_INCREMENT | Unique product identifier |

| name | VARCHAR(255) | NOT NULL | Product name |

| description | LONGTEXT | NULLABLE | Detailed product description |

| brand_id | BIGINT | FOREIGN KEY (brands.id) | Reference to brand |

| price | DECIMAL(10,2) | NOT NULL | Product price |

| stock_quantity | INT | NOT NULL, DEFAULT 0 | Available inventory count |

| created_at | TIMESTAMP | NOT NULL | Product creation timestamp |

| updated_at | TIMESTAMP | NOT NULL | Last update timestamp |

Entity 4: BRANDS

| Field | Data Type | Constraints | Description |

|-------|-----------|-------------|-------------|

| id | BIGINT | PRIMARY KEY, AUTO_INCREMENT | Unique brand identifier |

| name | VARCHAR(255) | NOT NULL, UNIQUE | Brand name |

| description | TEXT | NULLABLE | Brand description |

| created_at | TIMESTAMP | NOT NULL | Brand creation timestamp |

| updated_at | TIMESTAMP | NOT NULL | Last update timestamp |

Entity 5: ORDERS

| Field | Data Type | Constraints | Description |

|-------|-----------|-------------|-------------|

| order_id | BIGINT | PRIMARY KEY, AUTO_INCREMENT | Unique order identifier |

| customer_id | VARCHAR(255) | FOREIGN KEY (customers.customer_id) | Reference to customer |

| product_id | BIGINT | FOREIGN KEY (products.product_id) | Reference to product |

| admin_id | BIGINT | FOREIGN KEY (admins.admin_id) | Assigned admin |

| quantity | INT | DEFAULT 1 | Quantity ordered |

| total_price | DECIMAL(10,2) | NOT NULL | Order total amount |

| payment_method | ENUM('cash','card','gcash','paypal') | DEFAULT 'cash' | Payment method |

| status | ENUM('pending','processing','shipped','completed','cancelled') | DEFAULT 'pending' | Order status |

| order_date | TIMESTAMP | NOT NULL | Order placement timestamp |

| cancelled_at | TIMESTAMP | NULLABLE | Cancellation timestamp | |

completed_at | TIMESTAMP | NULLABLE | Completion timestamp |

| created_at | TIMESTAMP | NOT NULL | Record creation timestamp |

| updated_at | TIMESTAMP | NOT NULL | Last update timestamp |

Entity 6: ORDER_ITEMS

Field	Data Type	Constraints	Description
order_item_id	BIGINT	PRIMARY KEY, AUTO_INCREMENT	Unique order item identifier
order_id	BIGINT	FOREIGN KEY (orders.order_id)	Reference to order
product_id	BIGINT	FOREIGN KEY (products.product_id)	Reference to product
quantity	INT	NOT NULL	Item quantity ordered
unit_price	DECIMAL(10,2)	NOT NULL	Price per unit
subtotal	DECIMAL(10,2)	NOT NULL	Total for this line item
created_at	TIMESTAMP	NOT NULL	Record creation timestamp
updated_at	TIMESTAMP	NOT NULL	Last update timestamp

Entity 7: VOUCHERS

Field	Data Type	Constraints	Description
voucher_id	BIGINT	PRIMARY KEY, AUTO_INCREMENT	Unique voucher identifier
code	VARCHAR(255)	NOT NULL, UNIQUE	Voucher code (e.g., SUMMER20)
admin_id	BIGINT	FOREIGN KEY (admins.admin_id), NULLABLE	Reference to admin who created it
description	TEXT	NULLABLE	Voucher description
discount_type	ENUM('percentage','fixed')	NOT NULL	Type of discount
discount_value	DECIMAL(10,2)	NOT NULL	Discount amount or percentage
start_date	DATE	NOT NULL	Voucher validity start date
end_date	DATE	NOT NULL	Voucher validity end date
usage_limit	INT	NULLABLE	Maximum uses allowed
usage_count	INT	DEFAULT 0	Current usage count

| status | ENUM('active','inactive','expired') | DEFAULT 'active' | Voucher status |

| created_at | TIMESTAMP | NOT NULL | Voucher creation timestamp |

| updated_at | TIMESTAMP | NOT NULL | Last update timestamp |

Entity 8: USER_VOUCHERS

| Field | Data Type | Constraints | Description |

|-------|-----------|-------------|-------------|

| id | BIGINT | PRIMARY KEY, AUTO_INCREMENT | Unique record identifier |

| user_id | BIGINT | FOREIGN KEY (users.id) | Reference to user |

| voucher_id | BIGINT | FOREIGN KEY (vouchers.id) | Reference to voucher |

| is_used | BOOLEAN | DEFAULT false | Whether voucher has been used |

| used_at | TIMESTAMP | NULLABLE | Timestamp of voucher use |

| created_at | TIMESTAMP | NOT NULL | Distribution timestamp |

| updated_at | TIMESTAMP | NOT NULL | Last update timestamp |

Entity 9: VOUCHER_USAGE

| Field | Data Type | Constraints | Description |

|-------|-----------|-------------|-------------|

| id | BIGINT | PRIMARY KEY, AUTO_INCREMENT | Unique usage record identifier |

| voucher_id | BIGINT | FOREIGN KEY (vouchers.id) | Reference to voucher |

| user_id | BIGINT | FOREIGN KEY (users.id) | Reference to user |

| order_id | BIGINT | FOREIGN KEY (orders.id) | Reference to order |

| used_at | TIMESTAMP | NOT NULL | Timestamp of usage |

| created_at | TIMESTAMP | NOT NULL | Record creation timestamp |

Entity 10: PAYMENTS

Field	Data Type	Constraints	Description
id	BIGINT	PRIMARY KEY, AUTO_INCREMENT	Unique payment identifier
order_id	BIGINT	FOREIGN KEY (orders.id)	Reference to order
amount	DECIMAL(10,2)	NOT NULL	Payment amount
payment_method	VARCHAR(50)	NOT NULL	Payment method (credit_card, bank_transfer, etc.)
status	ENUM('pending','completed','failed')	DEFAULT 'pending'	Payment status
transaction_id	VARCHAR(255)	NULLABLE	External transaction reference
created_at	TIMESTAMP	NOT NULL	Payment timestamp
updated_at	TIMESTAMP	NOT NULL	Last update timestamp

Entity 11: CUSTOMERS

Field	Data Type	Constraints	Description
id	BIGINT	PRIMARY KEY, AUTO_INCREMENT	Unique customer identifier
user_id	BIGINT	FOREIGN KEY (users.id)	Reference to user account
first_name	VARCHAR(255)	NOT NULL	Customer's first name
last_name	VARCHAR(255)	NOT NULL	Customer's last name
phone	VARCHAR(20)	NULLABLE	Customer's phone number
address	TEXT	NULLABLE	Physical address

| created_at | TIMESTAMP | NOT NULL | Record creation timestamp |

| updated_at | TIMESTAMP | NOT NULL | Last update timestamp |

Entity 12: SHIPPING

Field	Data Type	Constraints	Description
id	BIGINT	PRIMARY KEY, AUTO_INCREMENT	Unique shipping record identifier
order_id	BIGINT	FOREIGN KEY (orders.id)	Reference to order
address	TEXT	NOT NULL	Shipping address
city	VARCHAR(100)	NOT NULL	City
state	VARCHAR(100)	NULLABLE	State/Province
postal_code	VARCHAR(20)	NOT NULL	Postal code
tracking_number	VARCHAR(100)	NULLABLE	Carrier tracking number
status	ENUM('pending','shipped','delivered','returned')	DEFAULT 'pending'	Shipping status
created_at	TIMESTAMP	NOT NULL	Record creation timestamp
updated_at	TIMESTAMP	NOT NULL	Last update timestamp

Entity 13: RETURN_REQUESTS

Field	Data Type	Constraints	Description
id	BIGINT	PRIMARY KEY, AUTO_INCREMENT	Unique return request identifier
user_id	BIGINT	FOREIGN KEY (users.id)	Reference to customer
order_id	BIGINT	FOREIGN KEY (orders.id)	Reference to original order

| status | ENUM | DEFAULT 'pending' | Return status (pending, approved, rejected, refunded) |

| reason | TEXT | NULLABLE | Reason for return |

| notes | TEXT | NULLABLE | Additional customer notes |

| refund_amount | DECIMAL(10,2) | NULLABLE | Calculated refund amount |

| processed_by | BIGINT | FOREIGN KEY (admins.admin_id) | Admin who processed return |

| processed_at | TIMESTAMP | NULLABLE | Timestamp of admin action |

| created_at | TIMESTAMP | NOT NULL | Return request creation timestamp | |

updated_at | TIMESTAMP | NOT NULL | Last update timestamp |

Entity 14: RETURN_REQUEST_ITEMS

| Field | Data Type | Constraints | Description |

|-------|-----------|-------------|-------------|

| return_request_item_id | BIGINT | PRIMARY KEY, AUTO_INCREMENT | Unique return item identifier |

| request_id | BIGINT | FOREIGN KEY (return_requests.request_id) | Reference to return request |

| order_item_id | BIGINT | FOREIGN KEY (order_items.order_item_id) | Reference to original order item |

| product_id | BIGINT | FOREIGN KEY (products.product_id), NULLABLE | Reference to product |

| quantity | INT | NOT NULL, DEFAULT 1 | Quantity being returned |

| refund_amount | DECIMAL(10,2) | NULLABLE | Refund amount for this item |

| status | ENUM('pending','approved','rejected','refunded') | DEFAULT 'pending' | Status of individual return item |

| notes | TEXT | NULLABLE | Additional notes for this return item |

| requested_at | TIMESTAMP | NOT NULL, DEFAULT CURRENT_TIMESTAMP | Timestamp when return was requested |

| created_at | TIMESTAMP | NOT NULL | Record creation timestamp |

| updated_at | TIMESTAMP | NOT NULL | Last update timestamp |

4.3 Relationships

Describe the relationships between entities (One-to-One, One-to-Many, Many-to-Many).

Relationship 1: CUSTOMERS ← One-to-Many → ORDERS

Description: Each customer can place multiple orders. Orders reference customers via customer_id (VARCHAR string primary key). Enables order history tracking per customer.

Relationship 2: ORDERS ← One-to-Many → ORDER_ITEMS

Description: Each order contains one or more items. OrderItems act as a junction table linking orders to products with quantity and pricing information.

Relationship 3: PRODUCTS ← One-to-Many → ORDER_ITEMS

Description: Each product can appear in multiple orders. This tracks which products have been sold and their sales history.

Relationship 4: BRANDS ← One-to-Many → PRODUCTS

Description: Each brand has multiple products. This enables product categorization and brandbased filtering.

Relationship 5: ADMINS ← One-to-Many → ORDERS

Description: Each admin can be assigned to manage multiple orders. Tracks which admin is responsible for order fulfillment.

Relationship 6: VOUCHERS ← One-to-Many → USER_VOUCHERS

Description: Each voucher is distributed to multiple users. This enables bulk promotional distribution while tracking individual availability status.

Relationship 7: USERS ← One-to-Many → USER_VOUCHERS

Description: Each user can receive multiple vouchers. USER_VOUCHERS maintains unique constraint on (user_id, voucher_id) to prevent duplicate distributions.

Relationship 8: VOUCHERS ← One-to-Many → VOUCHER_USAGES

Description: Each voucher can be used in multiple transactions. Tracks actual usage history separately from distribution (USER_VOUCHERS).

Relationship 9: USERS ← One-to-Many → VOUCHER_USAGES

Description: Each user's voucher usage is tracked in VOUCHER_USAGES. Maintains record of when and where vouchers are applied.

Relationship 10: PRODUCTS ← One-to-Many → VOUCHER_USAGES

Description: Each product can have vouchers applied across multiple usage records. Enables tracking of promotion effectiveness per product.

Relationship 11: ADMINS ← One-to-Many → VOUCHERS

Description: Admins can create and manage vouchers. admin_id field is nullable, allowing systemgenerated vouchers without admin assignment.

Relationship 12: ORDERS ← One-to-Many → PAYMENTS

Description: Each order has associated payment record(s). PAYMENTS tracks order payment status, method, and transaction details.

Relationship 13: CUSTOMERS ← One-to-Many → PAYMENTS

Description: Customer payment history is tracked. PAYMENTS references both order and customer for payment reconciliation.

Relationship 14: ORDERS ← One-to-Many → SHIPPING

Description: Each order has one shipping record containing delivery address, tracking, and delivery status information.

Relationship 15: ORDERS ← One-to-Many → RETURN_REQUESTS

Description: Orders can have multiple return requests. Returns reference the original order for validation and refund processing.

Relationship 16: CUSTOMERS ← One-to-Many → RETURN_REQUESTS

Description: Customer-initiated returns are tracked via customer_id. Enables return request history per customer.

Relationship 17: PRODUCTS ← One-to-Many → RETURN_REQUESTS

Description: Return requests reference specific products being returned. Tracks return patterns per product.

Relationship 18: RETURN_REQUESTS ← One-to-Many → RETURN_REQUEST_ITEMS

Description: Each return request contains multiple items being returned. Mirrors ORDER_ITEMS structure for detailed return tracking.

Relationship 19: ORDER_ITEMS ← One-to-Many → RETURN_REQUEST_ITEMS

Description: Return request items reference original order items. Maintains link between purchase and return for refund calculation.

Relationship 20: PRODUCTS ← One-to-Many → RETURN_REQUEST_ITEMS

Description: Product-level return tracking. Enables analytics on which products have higher return rates.

4.4 Database Normalization

Explain how your database follows normalization principles (1NF, 2NF, 3NF).

First Normal Form (1NF) Compliance:

Atomic Values: All fields contain single, indivisible values

- Example: Address stored as separate fields (address, city, state, postal_code) rather than single concatenated field
- No repeating groups; related items stored in separate tables

Primary Keys: Every entity has a unique primary key

- User IDs, Product IDs, Order IDs, etc. ensure unique identification

Second Normal Form (2NF) Compliance:

Full Functional Dependency: All non-key attributes fully depend on the entire primary key

- In ORDER_ITEMS: product_id and quantity both depend on the entire composite relationship (order_id + product_id)
- In USER_VOUCHERS: voucher status depends on the combination of user_id and voucher_id

No Partial Dependencies: Non-key attributes don't depend on part of a composite key

- Price at purchase stored in ORDER_ITEMS, not derived from PRODUCTS - Ensures historical accuracy of pricing

Third Normal Form (3NF) Compliance:

No Transitive Dependencies: Non-key attributes don't depend on other non-key attributes

- Product price doesn't depend on brand; product info stored in PRODUCTS table
- Customer name doesn't depend on order info; CUSTOMERS table separate from ORDERS - Eliminates data redundancy

---

5. Application Features & Functionality

5.1 Feature 1: Order Management & Order History

Description: Customers can view their complete order history, access detailed information about each order, track order status, and monitor order progress through the fulfillment.

Functionality:

- Order History Dashboard: View paginated list of all customer orders (10 per page)
- Order Statistics: Display total orders, total spent, pending orders in statistics cards
- Order Details: Click order to view full details including:
    - -Order ID and placement date
    - - All items purchased with quantities and prices
    - - Order total and payment method
    - - Shipping address and status
    - - Estimated delivery date

- Status Tracking: Visual status indicators (pending, completed, cancelled)
- Status Filtering: Filter orders by status for quick lookup
- Order Actions: View details, request return, track shipment

Implementation Details: Routes:**

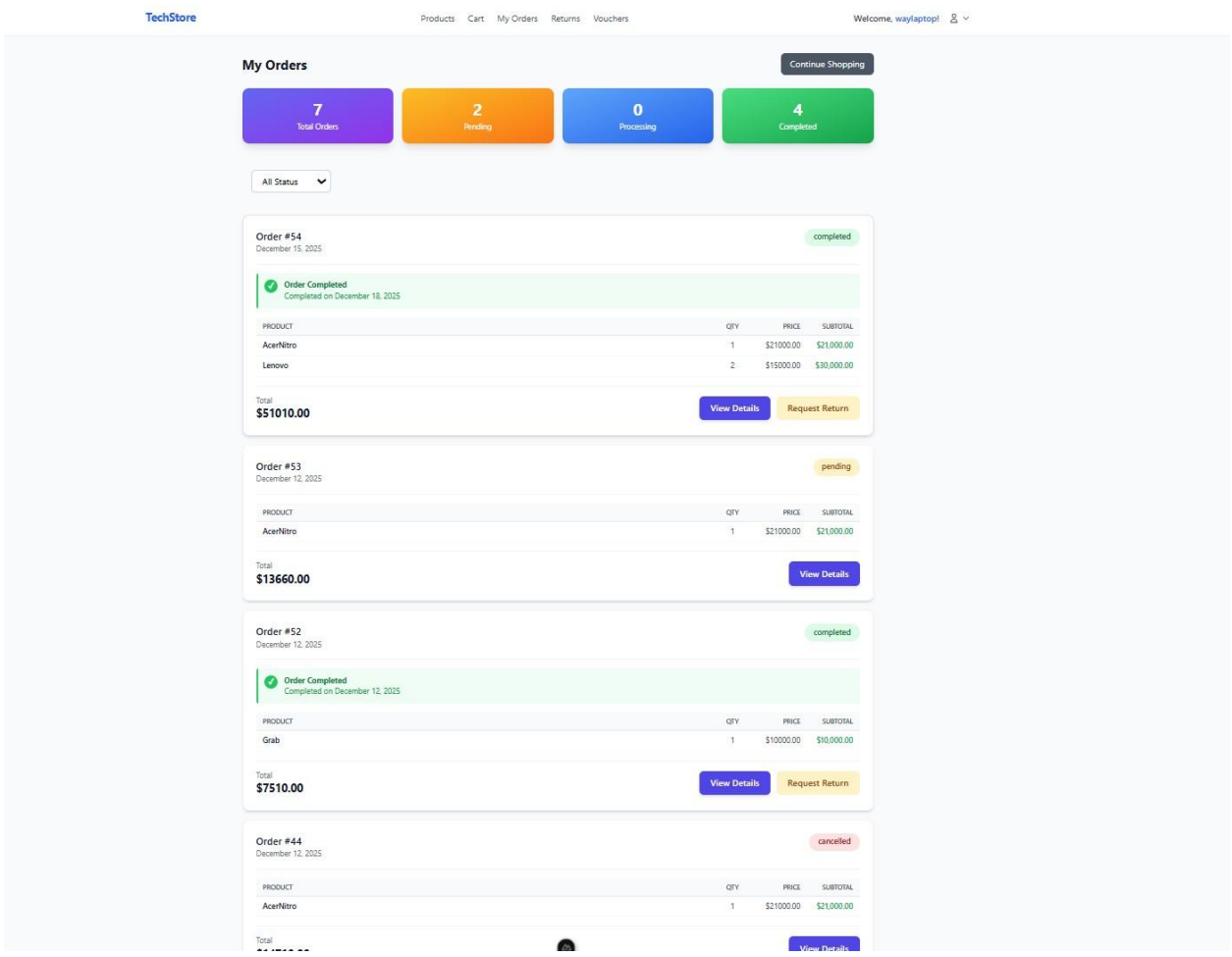
- GET `/orders` - Dashboard
- GET `/orders/{orderId}` - Details
- GET `/orders-stats` - Statistics JSON
- **Controller:**`UserOrderItemController@index()`, `show()`, `stats()`
- **Authorization:** Users can only see their own orders

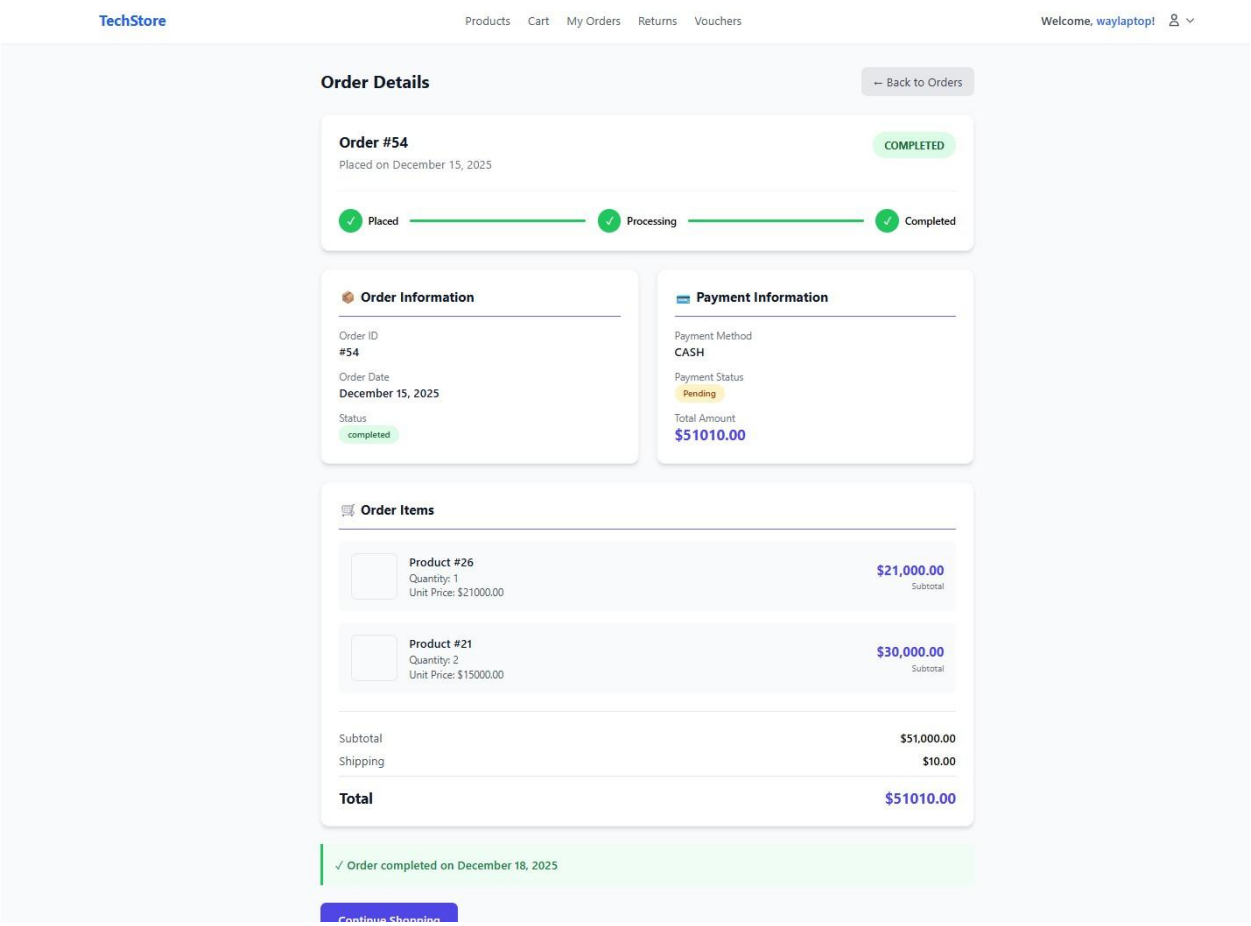
-**Pagination:** Laravel pagination (10 items per page)

-**Relationships:** Eager loading to prevent N+1 queries

-**Performance:** Optimized queries with selects and joins

Screenshots:


TechStore
Products
Cart
My Orders
Returns
Vouchers
Welcome, waylaptop!
My Orders
Continue Shopping
7
Total Orders
2
Pending
0
Processing
4
Completed
All Status
Order #54
December 15, 2025
completed
Order Completed
Completed on December 18, 2025
PRODUCT
QTY
PRICE
SUBTOTAL
AcerNitro
1
$21000.00
$21,000.00
Lenovo
2
$15000.00
$30,000.00
Total
$51010.00
View Details
Request Return
Order #53
December 12, 2025
pending
PRODUCT
QTY
PRICE
SUBTOTAL
AcerNitro
1
$21000.00
$21,000.00
Total
$13660.00
View Details
Order #52
December 12, 2025
completed
Order Completed
Completed on December 12, 2025
PRODUCT
QTY
PRICE
SUBTOTAL
Grab
1
$10000.00
$10,000.00
Total
$7510.00
View Details
Request Return
Order #44
December 12, 2025
cancelled
PRODUCT
QTY
PRICE
SUBTOTAL
AcerNitro
1
$21000.00
$21,000.00
Total



5.2 Feature 2: Return Request Management

Description: Provides a complete return workflow where customers can initiate return requests, admins can review and approve/reject requests, and refunds are automatically calculated and tracked.

Functionality:

- Customer Side:
    - Return Dashboard: View all return requests with status indicators and filtering ○ Request Creation: Submit new return requests with:

- Order selection from past purchases
- Itemized selection (select specific items from order)
- Quantity specification
- Reason selection (Defective, Wrong Item, Not as Described, Changed Mind, Other) - Optional notes ○ Auto Calculation: System automatically calculates refund amount

- Status Tracking:** Real-time status updates (pending, approved, rejected, refunded) ○ Timeline View:** See history of status changes with timestamps

- Admin – side:
  - Returns Dashboard: List all return requests with filtering by status ○ Review Details: View full return information including:

- Customer details
- Original order information
- Items being returned with reasons
- Calculated refund amount ○ Approval Workflow: Approve or reject returns with:
- Approval button with optional notes
- Rejection button with required reason ○ Audit Trail: Track which admin processed each return and when ○ Statistics: Dashboard cards showing total, pending, approved, rejected, refunded counts

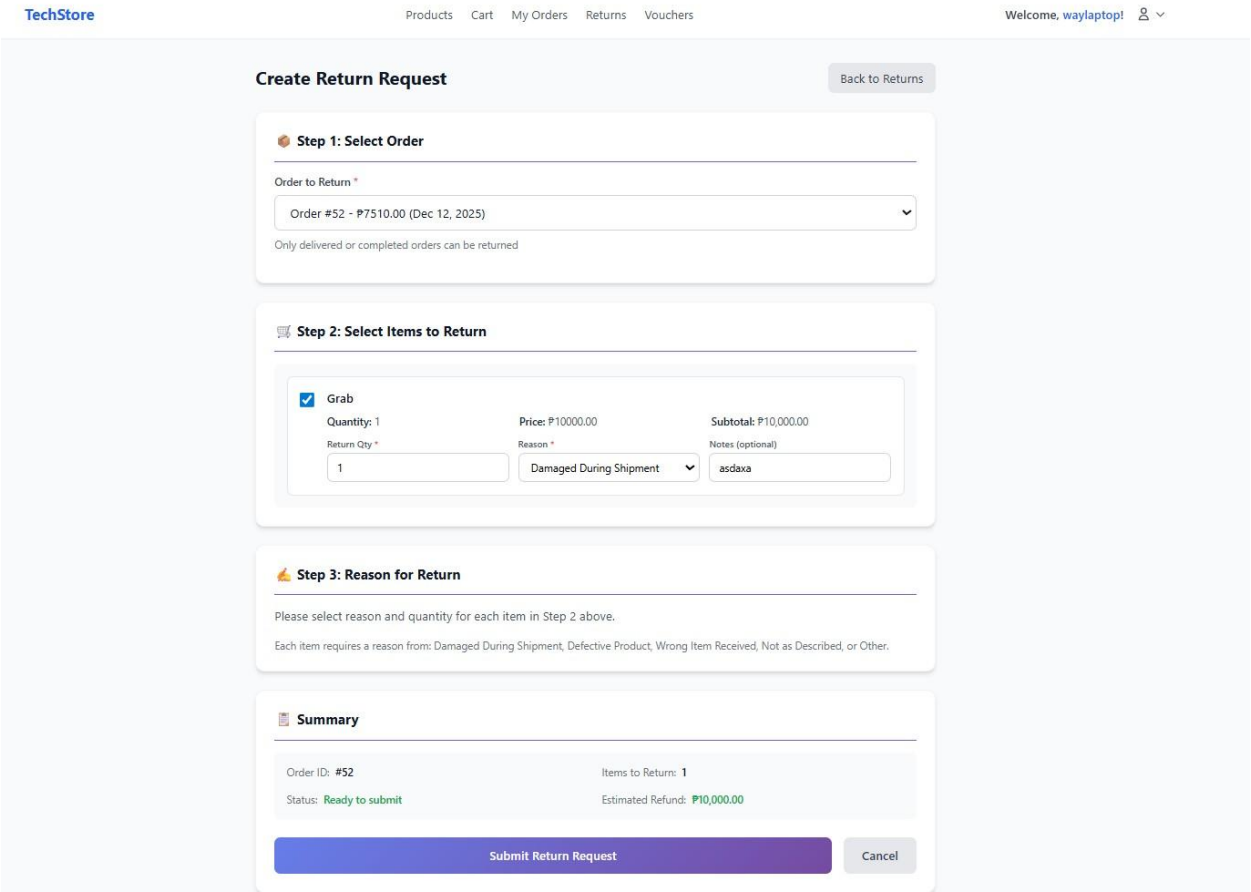
Implementation Details: Implementation Details:

- Controllers:
- `UserReturnRequestController` (customer operations)
- `AdminReturnController` (admin operations)
- Routes
- GET `/returns` - Customer dashboard
- GET `/returns/create` - Form
- POST `/returns` - Submit request
- GET `/returns/{id}` - Details
- GET `/admin/returns` - Admin dashboard
- PATCH `/admin/returns/{id}/approve` - Approve
- PATCH `/admin/returns/{id}/reject` - Reject
- Models: ReturnRequest, ReturnRequestItem with relationships
- Validation: Ensures items belong to customer's orders, quantity limits

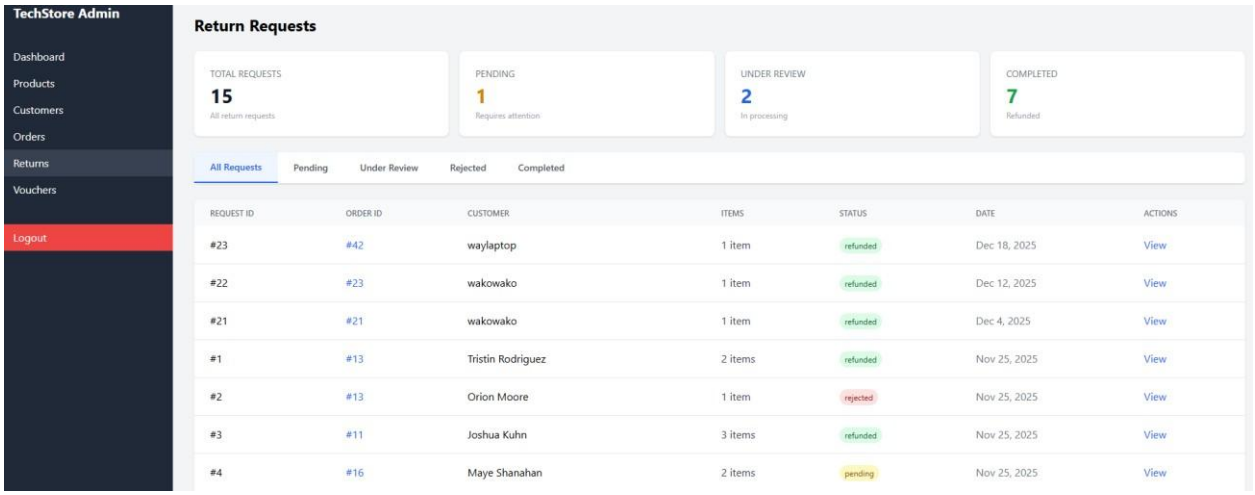
- Calculation: Refund calculated based on OrderItem prices
- Authorization: Row-level security ensures users see only their returns

Screenshots:

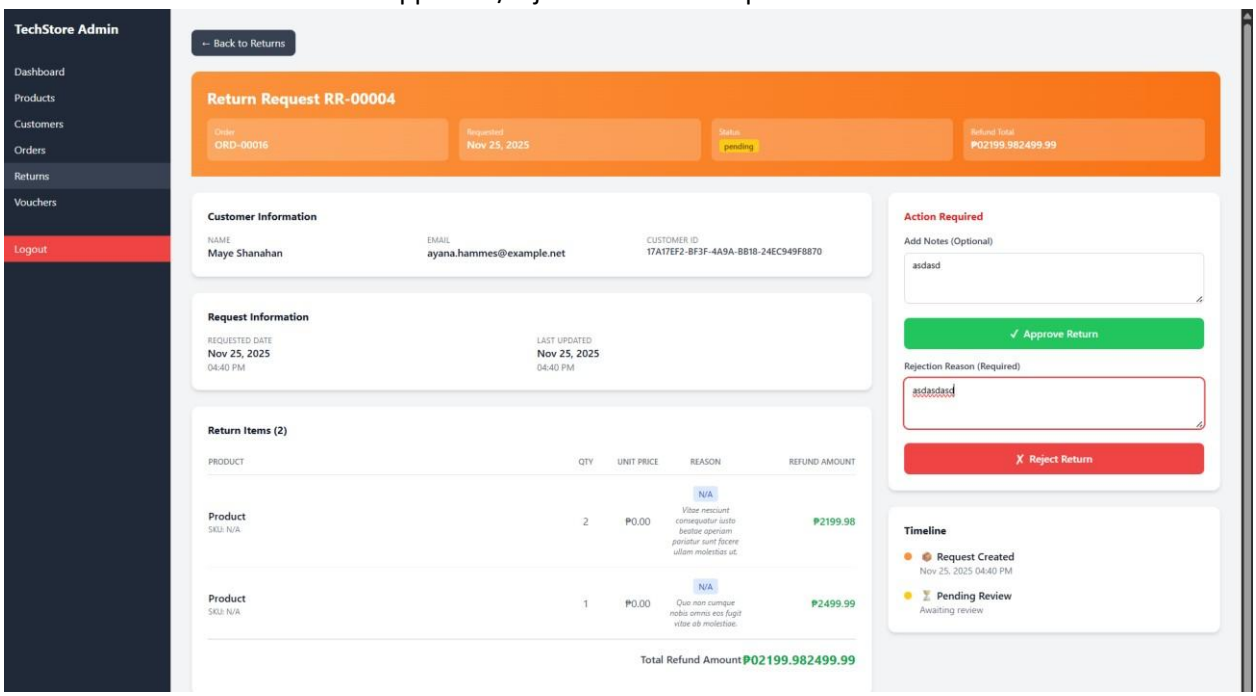
in this feature customer can create a return request



In this feature admin can view the history of the return request



In this feature admin can review approved/reject the return request



---

6. Security & Error Handling

6.1 Security Measures Implemented

Describe the security features implemented in your application.

- **Authentication:**

- User Registration & Login: Email-based authentication with password hashing using bcrypt
- Admin Authentication: Separate admin login with role-based access control ○ Session Management: Laravel sessions with secure cookies ○ Sanctum Tokens: API authentication using Laravel Sanctum Bearer tokens ○ Token Generation: Secure token generation for API access: `$user->createToken('api')->plainTextToken`

- **Authorization:**
  - Role-Based Access Control (RBAC): - Customer role: Can view own orders, create returns, apply vouchers - Admin role: Can manage orders, process returns, create vouchers
  - Route Protection:** - Web routes: Protected with `auth` middleware - Admin routes: Protected with `auth:admin` middleware - API routes: Protected with `auth:sanctum` middleware ○ Row-Level Security:** - Customers can only view their own orders, returns, and vouchers - Admins can view all data but process specific items - Query scoping ensures users don't access other users' data
- **Input Validation:**
  - Server-Side Validation: - All form inputs validated in controllers before database operations - Comprehensive validation rules for each entity - Custom validation rules for complex constraints (date ranges, numeric ranges)
  - Validation Examples: Orders: user_id must belong to authenticated user Returns: order_id and items must belong to user's orders Vouchers: code must be unique, dates must be valid, value must be positive Payments: amount must match order total
  - Client-Side Validation: - Form field validation in Vue.js/Blade - Provides immediate user feedback - Prevents unnecessary server requests
  - Sanitization: - HTML entities encoded in output - Blade template escaping: `{{ $variable }}` - User-provided data sanitized before storage
- **Data Protection:**
  - Password Hashing:** bcrypt algorithm with automatic salting ○ Encryption:** - Sensitive configuration in `.env` file (not in code) - Payment tokens encrypted in database - Session data encrypted by default in Laravel
  - HTTPS Enforcement:** - All communications over SSL/TLS in production - Secure cookie flag set: `HTTP_ONLY`, `SECURE`, `SAME_SITE

- Database Security:** - Parameterized queries via Eloquent ORM - No raw SQL concatenation - Database passwords not in version control

- **Protection Against Vulnerabilities:**
  - CSRF Protection:** - CSRF tokens generated for all forms - Token validation on form submission - Token included in all state-changing requests (POST, PUT, DELETE, PATCH) - Protection automatically applied by Laravel middleware ○ SQL Injection Prevention:** - Eloquent ORM prevents SQL injection through parameterized queries - Example: `Order::where('user_id', $userId)` uses bound parameters - No raw SQL queries without proper binding
  - XSS (Cross-Site Scripting) Prevention:** - Blade template escaping: `{{ $data }}` automatically escapes HTML - Raw output only with `{!! $data !!}` when intentional - User input displayed through escaped output - JavaScript code never built from user input
  - Authentication Bypass Prevention:** - Password reset tokens with expiration (1 hour default) - Session invalidation on logout - Brute force protection (optional, can be added) - Account lockout after failed attempts (optional enhancement)
  - Authorization Bypass Prevention:** - Policies define what users can do (optional, currently using middleware) - Request validation ensures user owns the resource - Admin check before sensitive operations - Query scoping prevents accessing other users' data

6.2 Error Handling

Describe how your application handles and displays errors to users.

Application-Level Error Handling:

Exception Handling Pipeline

Exception occurs

↓

Laravel exception handler catches it

↓

Logs error with full context

↓

Generates appropriate HTTP response

├─ Production: Generic error message

└─ Development: Detailed error information

↓

Returns response to user

Error Response Examples:

Validation Errors (422 Unprocessable Entity):

json

```
{
  "message": "Validation failed",
  "errors": {
    "email": ["The email field must be a valid email."],
    "password": ["The password must be at least 8 characters."]
  }
}
```

Authorization Errors (403 Forbidden):

json

```
{
  "message": "This action is unauthorized."
}
```

Not Found Errors (404):

json

```
{
  "message": "Order not found"
}
```

Server Errors (500)

Production: "Something went wrong. Please try again later."

Development: Full stack trace with error details

---

7. Installation & Setup Instructions

7.1 Prerequisites

List all software and tools required to run your application.

- Backend Requirements:

```
- **PHP 8.2 or higher** - Laravel 12 requires PHP 8.2+
- **Composer 2.0+** - PHP dependency manager for Laravel
- **Laravel 12** - Backend framework
- **Node.js 18+ and npm 9+** - JavaScript runtime for asset compilation
- **Database System** - One of:
  - SQLite (built-in, suitable for development)
  - MySQL 8.0 or higher
  - PostgreSQL 12 or higher
- **Git 2.0+** - Version control system
```

- Frontend Requirements:

```
- **Node.js 18+** - JavaScript runtime
- **npm 9+ or Yarn** - Package manager for Node.js
- **Nuxt.js 3.x** - Vue.js framework for the public website
- **Vue.js 3.x** - Used in Blade components and admin dashboard
```

7.2 Installation Steps

Provide step-by-step instructions to set up and run your application.

1. Clone the repository: git clone https://github.com/daromaguera/ITE-18-1st-ActivitySY2526.git
2. Navigate to the project directory: cd Techstore
3. Install dependencies: [command for your technology stack] ```bash

   # Install PHP dependencies
   composer install

Copy the environment configuration file
cp .env.example .env

Generate application key
php artisan key:generate

```

4. Configure environment variables:

   Edit the `.env` file in the project root and configure the following:

   Database Configuration:**
   ```env
   # Database connection type (sqlite, mysql, pgsql) DB_CONNECTION=sqlite  #
   For SQLite, ensure database file exists  # For MySQL:
   DB_HOST=localhost
   DB_PORT=3306
   DB_DATABASE=techstore
   DB_USERNAME=root
   DB_PASSWORD=your_password

   ```

   Application Configuration:**
   ```env
   APP_NAME=TechStore
   APP_ENV=local                # local, staging, or production
   APP_DEBUG=true               # false in production
   APP_URL=http://localhost:8000
   FRONTEND_URL=http://localhost:3000
   ```

   Cache & Session:**
   ```env
   CACHE_DRIVER=file
   SESSION_DRIVER=cookie
   SESSION_LIFETIME=120         # In minutes
   ```

   Mail Configuration (for notifications):**

   ```env
   MAIL_MAILER=log              # Use 'log' for development
   MAIL_FROM_ADDRESS=noreply@techstore.local

```

Sanctum Configuration (API tokens):**
```env
SANCTUM_STATEFUL_DOMAINS=localhost:3000,127.0.0.1:3000 ```

5. Set up the database:

```bash
# Run migrations
php artisan migrate

# Seed database with sample data (optional)
 php artisan db:seed
```

For MySQL:**

```bash

# Create the database in MySQL  mysql -
u root -p
CREATE DATABASE techstore;
EXIT;

# Run migrations
php artisan migrate

# Seed database with sample data (optional)
 php artisan db:seed

```

Frontend Setup (Node.js Dependencies)**
```bash
# Install Node.js dependencies
 npm install
```

Compile Frontend Assets**

```bash
# Compile assets for development (with hot reload)    npm
run dev

# Or build for production
```

npm run build

```

6. Run the application:

Running the Application #### **Development Environment**

Open two terminal windows or tabs for concurrent execution:

Terminal 1 - Laravel Backend Server:**
```bash
# Start Laravel development server
  php artisan serve

# Server runs at: http://localhost:8000
# API endpoints accessible at: http://localhost:8000/api/...

```

Terminal 2 - Node.js Frontend Build (Vite):**
Nuxt.js Frontend (if using separate Nuxt app):

```bash cd nuxt-
techstore npm
install npm run
dev

# Nuxt frontend runs at: http://localhost:3000

```

7.3 Running the Application

Explain how to access and use the application once it's running.

Frontend: http://localhost:3000
Admin: http://localhost:8000/admin
Customer: http://localhost:8000/dashboard
API: http://localhost:8000/api/techstore/
```

---

8. Testing

8.1 Test Cases

Describe the test cases you performed to verify functionality.

| Test Case # | Test Name | Steps | Expected Result | Status |

|---|---|---|---|---|

| **1** | User Registration | Navigate to signup → Enter email/password → Click Register → Verify confirmation | User account created with hashed password, redirected to dashboard | PASS |

| **2** | User Login | Navigate to login → Enter credentials → Click Login → Check session/token | Sanctum token generated, user redirected to dashboard | PASS |

| **3** | Browse Products | Access catalog → View products → Filter by brand → Sort by price | All products displayed correctly, filters functional | PASS |

| **4** | View Product Details | Click on product → View details page → Check related items → Verify stock | All details display correctly, stock validation works | PASS |

| **5** | Add to Cart | Add product → Verify quantity → Check cart total → Test stock validation | Cart updated in real-time, localStorage/database persisted | PASS |

| **6** | Cart Persistence | Add items → Refresh page → Logout/login → Check cart contents | Cart retrieved from localStorage (anonymous) or database (authenticated) | PASS |

[Add more test cases as needed] **8.2**

Known Issues & Limitations

Document any known bugs, limitations, or future improvements.
Payment Gateway Integration**
- Issue: Payment processing not fully integrated with external payment providers

- Current Status: Mock payment flow implemented for testing
- Solution: Implement real payment gateway (Stripe, PayPal, GCash API) in production
- Impact: Orders can be created with "Paid" status, but actual payment processing not connected - Timeline:** Scheduled for Phase 2

Future Enhancements**

| Enhancement | Priority | Effort | Timeline |

|---|---|---|---|

| Payment Gateway Integration (Stripe, PayPal) | High | 3-4 days | Phase 2 |

---

9. Code Quality & Documentation

9.1 Code Structure

Describe your project's directory structure and organization. techstore/

├── app/ # Laravel application core

│ ├── Http/

│ │ ├── Controllers/ # Request handlers (ProductController, OrderController, etc.)

│ │ ├── Middleware/ # Authentication & authorization (auth:sanctum, auth:admin)

│ │ ├── Requests/ # Form request validation classes

│ │ └── Resources/ # API response transformations

│ ├── Models/ # Eloquent ORM models

│ │ ├── User.php # Customer user model

│ │ ├── Admin.php # Admin user model

│ │ ├── Product.php # Product catalog

│ │ ├── Order.php # Order transactions

├── OrderItem.php # Order line items

│ │

│ │

│ │

│ │ ├──

```
|   |
|   |
|   |
|
|
        ├── ReturnRequest.php     # Return management
        ├── ReturnRequestItem.php # Individual return items
              Voucher.php          # Promotional vouchers
        ├── UserVoucher.php       # User voucher distribution
        ├── VoucherUsage.php      # Voucher usage tracking
        ├── Payment.php           # Payment transactions
    |   ├── Customer.php          # Customer profile data
|   |   ├── Brand.php             # Product brands
|   |   └── Shipping.php          # Shipping information
|   ├── Services/                 # Business logic services
|   |   ├── OrderService.php      # Order processing logic
|   |   ├── ReturnService.php     # Return handling logic
|   |   ├── VoucherService.php    # Voucher distribution & validation
|   |   └── PaymentService.php    # Payment processing logic
|   ├── Notifications/            # Email & notification classes
|   └── Providers/                # Service providers for DI container
├── bootstrap/
|   ├── app.php                   # Application container setup
|   └── cache/                    # Runtime caches
├── config/                       # Configuration files
|   ├── app.php                   # Application configuration
|   |
|   |
|   |
|   |   ├──
```

```
|   |
|   |
|   |
|
|   ├── database.php            # Database connections
|   ├── sanctum.php             # API authentication config
|   ├── mail.php                # Email configuration
|   └── queue.php               # Background job configuration
├── database/
|   ├── migrations/             # Database schema migrations
        ├── *_create_users_table.php
        ├── *_create_products_table.php
        ├── *_create_orders_table.php
              *_create_order_items_table.php
        ├── *_create_return_requests_table.php
        ├── *_create_vouchers_table.php
        └── *_create_payments_table.php
    ├── seeders/                # Database seeders
    |   ├── DatabaseSeeder.php    # Master seeder
|   |   ├── UserSeeder.php        # Create test customers
|   |   ├── AdminSeeder.php       # Create admin accounts
|   |   ├── ProductSeeder.php     # Populate products & brands
|   |   ├── OrderSeeder.php       # Create sample orders
|   |   └── VoucherSeeder.php     # Create sample vouchers
|   └── database.sqlite           # SQLite development database

|   |
|   |
|   |
|   |   ├──
```

| |

| |

| |

|

|

├— nuxt-techstore/ # Nuxt.js frontend application

| ├— components/ # Reusable Vue components

| | ├— ProductCard.vue # Product display component

| | ├— CartItem.vue # Shopping cart item

| | ├— OrderCard.vue # Order display

| | └— Modal.vue # Reusable modal component

| ├— pages/ # Page components (auto-routed by Nuxt)

| | ├— index.vue # Home page

| | ├— products/

| | | ├— index.vue # Product catalog

| | | └— [id].vue # Product details |

| ├— cart.vue # Shopping cart

| | ├— checkout.vue # Checkout process ├—

dashboard/

| ├— orders.vue # Customer orders

| └— returns.vue # Return requests auth/

| |

| |

| |

| | ├—

```
|   |
|   |
|   |
|
|
        |   ├── login.vue        # Customer login
        |   └── register.vue     # Customer registration
        └── admin/               # Admin pages
    |       ├── dashboard.vue    # Admin dashboard |
     ├── orders.vue       # Order management
|   |       ├── returns.vue      # Return management
|   |       └── vouchers.vue     # Voucher management
|    ├── composables/            # Reusable Vue 3 composition functions
|   |    ├── useCart.ts          # Shopping cart logic
|   |    ├── useAuth.ts          # Authentication logic
|   |    └── useFetch.ts         # API calling utilities
|    ├── stores/                 # Pinia state management
|   |    ├── authStore.ts        # Authentication state
|   |    ├── cartStore.ts        # Cart state
|   |    └── userStore.ts        # User state
|    ├── layouts/
|   |    ├── default.vue         # Default layout
|   |    └── admin.vue           # Admin layout
|    ├── app.vue                 # Root component
|    ├── nuxt.config.ts          # Nuxt configuration
|    └── package.json            # Node.js dependencies
 ├── public/                    # Static files (images, icons)
|    ├── images/
```

```
|
|
|
|
|
|   |   ├── products/          # Product images
|   |   ├── brands/            # Brand logos
|   |   └── icons/             # UI icons
|   └── favicon.ico
├── resources/
    ├── css/                 # Global styles
    |   ├── app.css            # Application styles
    |   └── tailwind.css       # Tailwind CSS configuration
    ├── views/               # Blade templates
    |   ├── layouts/
|   |   |   ├── app.blade.php     # Main layout
|   |   |   └── admin.blade.php   # Admin layout
|   |   ├── auth/              # Auth templates
|   |   ├── dashboard/         # Customer dashboard templates
|   |   └── admin/             # Admin dashboard templates
|   └── js/
|       └── bootstrap.js       # Bootstrap JavaScript
├── routes/
|   ├── api.php               # API routes (/api/techstore/*)
|   ├── web.php               # Web routes (Blade rendering)
|   └── admin.php             # Admin routes (admin.php middleware)
├── storage/
|   ├── logs/
```

```
|
|
|
|
|
|   |   └── laravel.log          # Application logs
|   ├── app/                     # Application storage
|   ├── framework/               # Framework cache
|   |   ├── cache/
|   |   ├── sessions/
|   |   └── views/
|   └── uploads/                 # User uploads (if any)
├── tests/
|   ├── Feature/                 # Feature/integration tests
|   |   ├── OrderTest.php
    |   ├── ReturnRequestTest.php |
    └── VoucherTest.php
    ├── Unit/                    # Unit tests
    |   ├── Models/
    |   └── Services/
|   └── TestCase.php             # Base test class
├── .env.example                 # Environment variables template
├── .gitignore                   # Git ignore patterns
├── artisan                      # Laravel CLI tool
├── composer.json                # PHP dependencies
├── composer.lock                # Locked PHP versions
├── package.json                 # Node.js dependencies
├── package-lock.json            # Locked Node.js versions
```

│

│

│

│

│

├── vite.config.js # Vite build configuration

├── tailwind.config.js # Tailwind CSS configuration

├── phpunit.xml # PHPUnit test configuration

└── README.md # Project overview

9.2 Code Standards

Describe the coding standards and best practices followed.

- Naming conventions:
  Naming Conventions ○ ****PHP**

  (Laravel) Code:**

  ```php

  // Classes: PascalCase class

  OrderController { } class

  ReturnRequest { } class

  VoucherService { }

  // Methods & Functions: camelCase
  ```

```
public function createReturnRequest() { }
public function calculateRefund() { } private
function validateVoucher() { }

// Variables: camelCase
$totalPrice = 0;
$userOrders = [];
$isApproved = true;

// Constants: CONSTANT_CASE const
MAX_CART_ITEMS = 100; const
DEFAULT_DISCOUNT = 10;
define('COMMISSION_RATE', 0.15);

// Table names: plural, snake_case
create_table('users'); create_table('products');
create_table('order_items');
create_table('return_requests');

// Database columns: snake_case
$table->string('first_name');
$table->integer('stock_quantity'); $table->timestamp('created_at');
$table->decimal('total_price', 10, 2);

// Relationships: camelCase method names
public function orders() { } public function
returnRequests() { }
public function products() { }
```

```

- **JavaScript/Vue.js (Frontend) Code:**

```javascript
// Components: PascalCase
ProductCard.vue OrderHistory.vue
VoucherManager.vue

// Variables & Functions: camelCase
const userData = {}; function
calculateTotal() { } const fetchProducts
= async () => { }

// Constants: CONSTANT_CASE const API_BASE_URL =
'http://localhost:8000/api/techstore/'; const
DEFAULT_PAGE_SIZE = 10;

// CSS Classes: kebab-case
class="product-card" class="order-
item__header" class="btn btn-primary"

// Store modules: camelCase
authStore.ts cartStore.ts
userStore.ts
```

```

- Code style:
    - **PHP Code Style (PSR-12 Standard):****

```php
// Use type hints for parameters and return types public
function updateOrder(int $orderId, array $data): Order
{
    // Validate input
    $validated = validator()->validate($data);

    // Business logic
    $order = Order::findOrFail($orderId);
    $order->update($validated);

    return $order;
}

// Laravel conventions
// - Use dependency injection in constructors
// - Type hints for all parameters
// - Meaningful variable names
// - Single responsibility methods

class OrderService
{
    public function __construct(
        protected OrderRepository $orders,
        protected PaymentService $payments
```

) {}

public function processOrder(array $data): Order

{

// Implementation

}

}

```

**JavaScript/Vue Code Style (ESLint + Prettier):**

```javascript

// Use TypeScript where possible

interface Product { id: number;

name: string; price: number;

}

// Vue 3 Composition API pattern

<script setup lang="ts"> import {

ref, computed } from 'vue';

const items = ref<Product[]>([]); const total = computed(() => items.value.reduce((sum,

item) => sum + item.price, 0));

const addItem = async (product: Product) => {

// Implementation

};

</script>

```
// Template formatting
<template>
  <div class="container">
    <header class="header">
      <h1>{{ title }}</h1>
    </header>

    <main class="main">
      <div v-if="items.length > 0" class="items-grid">
        <ProductCard   v-
for="item in items"
:key="item.id"
          :product="item"
          @add="addItem"
        />
      </div>
      <EmptyState v-else message="No items found" />
    </main>
  </div>
</template>
```

Configuration Files:

```javascript
// .eslintrc.json - ESLint configuration
{
  "env": {
```

```
    "browser": true,
    "es2021": true,
    "node": true
  },
  "extends": [
    "eslint:recommended",
    "plugin:vue/vue3-recommended",
    "prettier"
  ],
  "rules": {
    "no-unused-vars": "warn",
    "no-console": "warn",
    "vue/multi-word-component-names": "off"
  }
}

// .prettierrc - Prettier code formatter
{
  "semi": true,
  "singleQuote": true,
  "trailingComma": "es5",
  "printWidth": 100,
  "tabWidth": 2
}
```

Laravel Pint (PHP Formatter):

```bash
```

```
# Format all PHP files according to PSR-12
./vendor/bin/pint

# Format specific directory
./vendor/bin/pint app/Models
```

9.3 API Endpoints (if applicable)

Document all API endpoints if your application has a backend API.

Endpoint 1: [GET/POST/PUT/DELETE] /api/[resource]

Endpoint 1: User Registration**

```
POST /auth/register
```

- ****Description:**** Register a new customer account
- ****Authentication:**** None (public endpoint)
- ****Request:****

```json
{
  "name": "John Doe",
  "email": "john@example.com",
  "password": "password123",
  "password_confirmation": "password123"
}
```

- ****Response (201 Created):****

```json
```

```
{
  "message": "User registered successfully",
  "user": {
    "id": 1,
    "name": "John Doe",
    "email": "john@example.com",
    "created_at": "2025-12-22T10:30:00Z"
  },
  "token": "1|abc123xyz789..."
}
```

- **Status Codes:**
- `201 Created` - User registered successfully
- `422 Unprocessable Entity` - Validation failed (invalid email, weak password)
- `409 Conflict` - Email already registered

Endpoint 2: User Login

```
POST /auth/login
```

- **Description:** Authenticate user and receive API token
- **Authentication:** None (public endpoint)
- **Request:**

```json
{
  "email": "john@example.com",
```

```
  "password": "password123"
}
```

- **Response (200 OK):**

```json
{
  "message": "Login successful",
  "user": {
    "id": 1,
    "name": "John Doe",
    "email": "john@example.com"
  },
  "token": "1|abc123xyz789..."
}
```

- **Status Codes:**
- `200 OK` - Login successful
- `401 Unauthorized` - Invalid credentials
- `404 Not Found` - User not found

Endpoint 3: User Logout

```
POST /auth/logout
```

- **Description:** Logout user and invalidate current token
- **Authentication:** Required (Bearer token)

- **Request:** Empty body

- **Response (200 OK):**

  ```json

  {

    "message": "Logged out successfully"

  }

  ```

- **Status Codes:**

- `200 OK` - Logout successful

- `401 Unauthorized` - Invalid or missing token

Product Endpoints

Endpoint 4: List All Products

```

GET /products

```

- **Description:** Retrieve paginated list of all products with filtering and sorting options

- **Authentication:** Optional (public endpoint)

- **Query Parameters:**

  ```

  ?page=1               # Page number (default: 1)

  &per_page=10          # Items per page (default: 10)

  &search=laptop        # Search by product name

  &brand_id=5           # Filter by brand ID

  &sort=price           # Sort field (price, name, created_at)
  ```

&order=asc # Sort order (asc, desc)

&min_price=100 # Minimum price filter

&max_price=5000 # Maximum price filter

```

- **Request:** None
- **Response (200 OK):**

```json
{
  "data": [
    {
      "id": 1,
      "name": "MacBook Pro",
      "description": "Powerful laptop for professionals",
      "brand_id": 2,
      "brand_name": "Apple",
      "price": 1999.99,
      "stock_quantity": 50,
      "created_at": "2025-12-20T08:00:00Z",
      "updated_at": "2025-12-22T10:30:00Z"
    }
  ],
  "pagination": {
    "current_page": 1,
    "per_page": 10,
    "total": 150,
    "last_page": 15
  }
}
```
```

```

- **Status Codes:**

- `200 OK` - Products retrieved successfully

- `400 Bad Request` - Invalid query parameters

---

**Endpoint 5: Get Product Details**

```
GET /products/{id}
```

- **Description:** Retrieve detailed information for a specific product

- **Authentication:** Optional (public endpoint)

- **URL Parameters:**

- `id` (required) - Product ID

- **Request:** None

- **Response (200 OK):**

```json
{
  "id": 1,
  "name": "MacBook Pro 16",
  "description": "Powerful laptop with M3 Max chip",
  "brand": {
    "id": 2,
    "name": "Apple",
    "description": "Apple Inc."
  },
  "price": 2499.99,
```
```

"stock_quantity": 50,

"created_at": "2025-12-20T08:00:00Z",

"updated_at": "2025-12-22T10:30:00Z"

}

```

- **Status Codes:**

- `200 OK` - Product found

- `404 Not Found` - Product does not exist

---

#### **Order Endpoints**

**Endpoint 6: List User Orders**

```

GET /orders

```

- **Description:** Retrieve paginated list of authenticated user's orders

- **Authentication:** Required (Bearer token)

- **Query Parameters:**

```

?page=1 # Page number (default: 1)

&per_page=10 # Items per page (default: 10)

&status=completed # Filter by status (pending, completed, cancelled)

&sort=created_at # Sort field (created_at, total_price)

&order=desc # Sort order (asc, desc)

```

- **Request:** None
```

- **Response (200 OK):**

```json
{
  "data": [
    {
      "order_id": 101,
      "user_id": 1,
      "total_price": 2499.99,
      "payment_method": "card",
      "status": "completed",
      "order_date": "2025-12-20T14:30:00Z",
      "completed_at": "2025-12-21T09:00:00Z",
      "created_at": "2025-12-20T14:30:00Z"
    }
  ],
  "pagination": {
    "current_page": 1,
    "per_page": 10,
    "total": 25,
    "last_page": 3
  }
}
```

- **Status Codes:**

- `200 OK` - Orders retrieved successfully

- `401 Unauthorized` - Invalid or missing token

Endpoint 7: Get Order Details

```
GET /orders/{orderId}
```

- **Description:** Retrieve full details of a specific order including items - **Authentication:** Required (Bearer token)
- **URL Parameters:**
- `orderId` (required) - Order ID
- **Request:** None
- **Response (200 OK):**

```json
{
  "order_id": 101,
  "user_id": 1,
  "total_price": 2499.99,
  "payment_method": "card",
  "status": "completed",
  "order_date": "2025-12-20T14:30:00Z",
  "completed_at": "2025-12-21T09:00:00Z",
  "items": [
    {
      "order_item_id": 201,
      "product_id": 1,
      "product_name": "MacBook Pro 16",
      "quantity": 1,
      "unit_price": 2499.99,
      "subtotal": 2499.99
    }
```

```
    ],
    "shipping": {
      "address": "123 Main St",
      "city": "San Francisco",
      "state": "CA",
      "postal_code": "94102",
      "tracking_number": "1Z999AA10123456784",
      "status": "delivered"
    },
    "payment": {
      "id": 301,
      "amount": 2499.99,
      "payment_method": "credit_card",
      "status": "completed",
      "transaction_id": "txn_1234567890"
    }
  }
```

- **Status Codes:**
- `200 OK` - Order found
- `401 Unauthorized` - Invalid token
- `403 Forbidden` - User does not own this order
- `404 Not Found` - Order does not exist

Endpoint 8: Get Order Statistics

```
```

GET /orders-stats

```

- **Description:** Retrieve aggregate statistics about user's orders
- **Authentication:** Required (Bearer token)
- **Request:** None
- **Response (200 OK):**

  ```json
  {
   "total_orders": 25,
   "total_spent": 12450.50,  "pending_orders":
  3,
   "completed_orders": 20,
    "cancelled_orders": 2,
    "average_order_value": 498.02
  }
  ```

- **Status Codes:**
- `200 OK` - Statistics retrieved successfully
- `401 Unauthorized` - Invalid or missing token

Return Request Endpoints

Endpoint 9: Create Return Request

```

POST /return-requests

```
```

- **Description:** Submit a new return request for items from a completed order

- **Authentication:** Required (Bearer token)

- **Request:**

```json
{
  "order_id": 101,
  "items": [
    {
      "order_item_id": 201,
      "quantity": 1,
      "reason": "Defective"
    }
  ],
  "notes": "Screen has dead pixels, product is unusable"
}
```

- **Response (201 Created):**

```json
{
  "id": 1,
  "user_id": 1,
  "order_id": 101,
  "status": "pending",
  "refund_amount": 2499.99,
  "notes": "Screen has dead pixels, product is unusable",
  "created_at": "2025-12-22T10:30:00Z"
}
```

- **Status Codes:**

- `201 Created` - Return request created successfully

- `400 Bad Request` - Invalid request data

- `401 Unauthorized` - Invalid or missing token

- `403 Forbidden` - User does not own this order

- `404 Not Found` - Order or item not found

- `422 Unprocessable Entity` - Order not eligible for return (e.g., already completed)

Endpoint 10: List User Return Requests

```
GET /return-requests
```

- **Description:** Retrieve all return requests for authenticated user

- **Authentication:** Required (Bearer token)

- **Query Parameters:**

```
?page=1                # Page number (default: 1)
&per_page=10           # Items per page (default: 10)
&status=pending        # Filter by status (pending, approved, rejected, refunded)
```

- **Request:** None

- **Response (200 OK):**

```json
{
  "data": [
    {
      "id": 1,
```

```
      "order_id": 101,
      "status": "pending",
      "refund_amount": 2499.99,
      "created_at": "2025-12-22T10:30:00Z",
     "updated_at": "2025-12-22T10:30:00Z"
   }
   ],
   "pagination": {
    "current_page": 1,
    "per_page": 10,
    "total": 5,
   "last_page": 1
 }
 }
```


- **Status Codes:**
- `200 OK` - Return requests retrieved successfully
- `401 Unauthorized` - Invalid or missing token

Endpoint 11: Get Return Request Details

```
GET /return-requests/{id}
```

- **Description:** Retrieve full details of a specific return request
- **Authentication:** Required (Bearer token)
- **URL Parameters:**

- `id` (required) - Return request ID

- **Request:** None

- **Response (200 OK):**

  ```json
  {
    "id": 1,
    "order_id": 101,
    "user_id": 1,
    "status": "pending",
    "refund_amount": 2499.99,
    "notes": "Screen has dead pixels",
    "created_at": "2025-12-22T10:30:00Z",
    "updated_at": "2025-12-22T10:30:00Z",
    "items": [
      {
        "return_request_item_id": 10,
        "order_item_id": 201,
        "product_id": 1,
        "product_name": "MacBook Pro 16",
        "quantity": 1,
        "refund_amount": 2499.99,
        "status": "pending",
        "reason": "Defective"
      }
    ]
  }
  ```

- **Status Codes:**

- `200 OK` - Return request found

- `401 Unauthorized` - Invalid token

- `403 Forbidden` - User does not own this return request

- `404 Not Found` - Return request not found

Voucher Endpoints

Endpoint 12: Get User Vouchers

```
GET /vouchers
```

- **Description:** Retrieve all vouchers available to authenticated user

- **Authentication:** Required (Bearer token)

- **Query Parameters:**

  ```
  ?page=1                # Page number (default: 1)
  &per_page=10           # Items per page (default: 10)
  &status=active         # Filter by status (active, expired, used)
  ```

- **Request:** None

- **Response (200 OK):**

  ```json
  {
    "data": [
      {
        "id": 1,
  ```

```
      "code": "SUMMER20",
      "description": "Summer sale - 20% off",
      "discount_type": "percentage",
      "discount_value": 20,
      "start_date": "2025-06-01",
      "end_date": "2025-08-31",
      "usage_limit": 100,
      "is_used": false,
      "used_at": null
    }
  ],
  "pagination": {
    "current_page": 1,
    "per_page": 10,
    "total": 8,
    "last_page": 1
  }
}
```


- **Status Codes:**
- `200 OK` - Vouchers retrieved successfully
- `401 Unauthorized` - Invalid or missing token

Endpoint 13: Validate Voucher Code

```
POST /vouchers/apply
```

```

- **Description:** Validate a voucher code and calculate discount amount
- **Authentication:** Required (Bearer token)
- **Request:**

  ```json
  {
    "code": "SUMMER20",
    "cart_total": 1000.00
  }
  ```

- **Response (200 OK):**

  ```json
  {
    "valid": true,
    "code": "SUMMER20",
    "description": "Summer sale - 20% off",
    "discount_type": "percentage",
    "discount_value": 20,
    "discount_amount": 200.00,
    "new_total": 800.00
  }
  ```

- **Error Response (400 Bad Request):**

  ```json
  {
    "valid": false,
    "error": "Voucher has expired or is not valid",
    "code": "VOUCHER_EXPIRED"
```
```

}

```

- **Status Codes:**

- `200 OK` - Voucher is valid

- `400 Bad Request` - Voucher is invalid, expired, or already used

- `401 Unauthorized` - Invalid or missing token

- `404 Not Found` - Voucher code does not exist

---

---

## 10. References & Resources

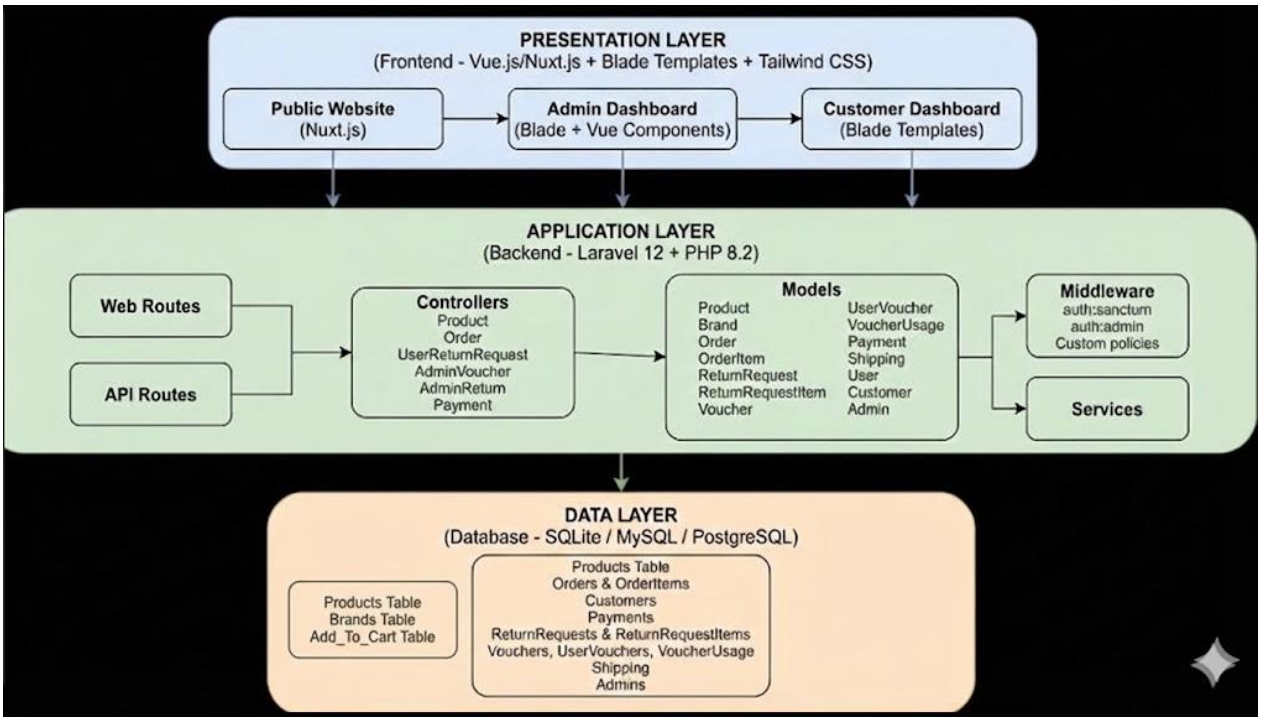
List any resources, tutorials, or documentation you used during development.

- Resource 1 - https://laravel.com/docs
- Resource 2 - https://nuxt.com/docs
- Resource 3 - https://vuejs.org/guide/introduction.html
- Resource 4 - https://tailwindcss.com/docs

---

## 11. Appendix

### 11.1 Additional Diagrams
```



11.2 Supplementary Information

[Include any additional information that doesn't fit in the sections above]

Student/Team Signature: _____ Date: _____